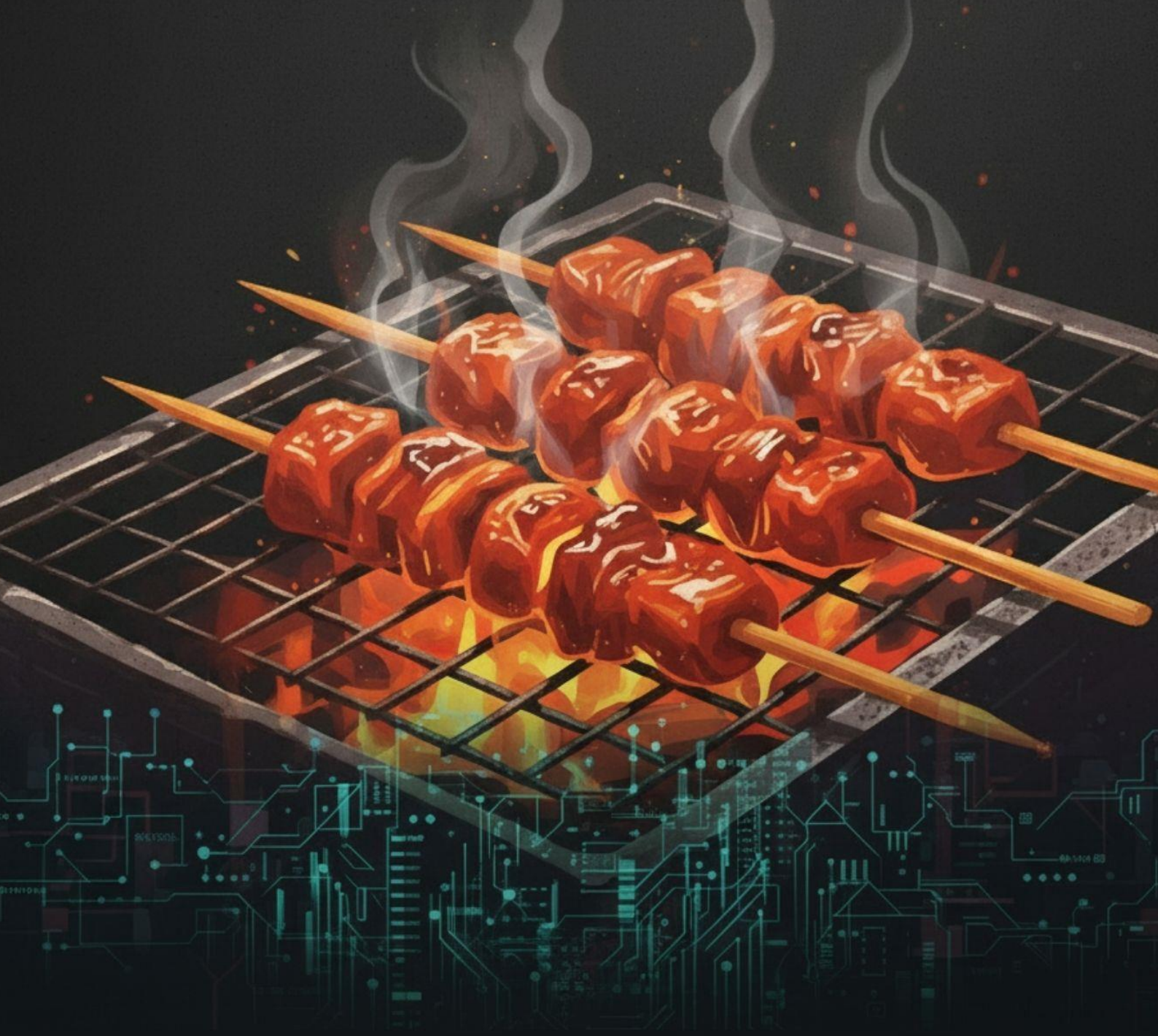


Perancangan Sistem Basis Data
Transaksi Penjualan

SATE REMBIGE

Disusun Oleh: Muhana Fawwas Sausan (1324073)

Dosen Pengampu : Finna Suroso, S.Kom., M.M.S.I.

DAFTAR ISI

DAFTAR ISI ..... **i**

DAFTAR GAMBAR ..... **iv**

DAFTAR TABEL ..... **vii**

BAB I PENDAHULUAN **8**

1.1 Latar Belakang 8

1.2 *Form* Transaksional 9

BAB II DESAIN BASIS DATA **11**

2.1 *Entity Relationship* Diagram (ERD) 11

2.2 *Logical Record Structure* (LRS) 14

2.3 *Physical Data Model* 18

BAB III *STRUCTURED QUERY LANGUAGE* (SQL) **22**

3.1 *Data Definition Language* (DDL) 22

3.1.1 Create Database 22

3.1.2 Show Database 23

3.1.3 Use Database 24

3.1.4 Create Table 24

3.1.5 Alter 28

3.1.6 Drop 30

3.1.7 Backup Database 31

3.1.8 Restore Database 33

3.2 *Data Manipulation Language* (DML) 35

3.2.1 *Insert* 36

3.2.2 *Select* 40

3.2.3 *Update* 42

3.2.4 *Delete* 43

3.2.5 Urutan Data 44

3.2.6 Agregat 44

3.2.7 *Between* 45

3.2.8 *Like* 45

3.2.9 Mengganti Nama *Field* Keluaran 46

3.2.10 Menambahkan Baris Teks *Field* Keluaran 46

3.2.11 Current_Date 47

3.2.12 Current_Time 47

3.3 *Data Control Language* (DCL) 48

3.3.1 *Grant* 48

3.3.2 *Revoke* 49

3.4 *Data Query Language* (DQL) 49

3.4.1 Menampilkan Detail Pembayaran dan Kasir Transaksi 49

3.4.2 Menampilkan Daftar Unik Total Nilai Transaksi 49

3.4.3 Menampilkan Data Kasir 50

3.4.4 Menampilkan Daftar Item dengan Harga di Atas 40.000 50

3.4.5 Menampilkan Jumlah Total Kuantitas Terjual per Item 51

3.4.6 Menampilkan Rekapitulasi Kuantitas Terjual Item > 2 51

3.4.7 Menampilkan Data Transaksi Berdasarkan Total Nilai Terbesar 52

3.5 *Transaction Control Language* (TCL) 53

3.5.1 *Commit* 53

3.5.2 *Rollback* 54

3.5.3 *Savepoint* 55

BAB IV IMPLEMENTASI 57

4.1 Struktur Basis Data 57
4.1.1 Struktur Tabel Item 57
4.1.2 Struktur Tabel Kasir 57
4.1.3 Struktur Tabel Konsumen 58
4.1.4 Struktur Tabel Transaksi 58
4.1.5 Struktur Tabel Detail_Transaksi 59
4.2 Antarmuka Sistem 59
4.2.1 Halaman Index 60
4.2.2 Halaman Login 62
4.2.3 Halaman Dashboard 63
4.2.4 Halaman Master Kasir 64
4.2.5 Halaman Master Konsumen 66
4.2.6 Halaman Master Item 68
4.2.7 Halaman Transaksi 71
4.2.8 Halaman Detail Transaksi 74
4.2.8 Halaman Cetak 76
BAB V PENUTUP 79
5.1 Kesimpulan 79
5.2 Saran 79
DAFTAR PUSTAKA 80

DAFTAR GAMBAR

Gambar 1. 1 Form transaksional sate rembige 10
Gambar 2. 1 ERD 12
Gambar 2. 2 LRS 17
Gambar 2. 3 Physical data model 20
Gambar 2. 4 Sub menu physical data model 21
Gambar 3. 1 Hasil create database 23
Gambar 3. 2 Hasil show database 23
Gambar 3. 3 Hasil use database 24
Gambar 3. 4 Hasil create table kasir 25
Gambar 3. 5 Hasil create table konsumen 25
Gambar 3. 6 Hasil create table item 26
Gambar 3. 7 Hasil create table transaksi 27
Gambar 3. 8 Hasil create table detail_transaksi 28
Gambar 3. 9 Hasil alter mengubah nama field 28
Gambar 3. 10 Hasil alter mengubah tipe data 29
Gambar 3. 11 Hasil alter menambah field 29
Gambar 3. 12 Hasil alter menghapus field 30
Gambar 3. 13 Seluruh table di database sate_rembige 30
Gambar 3. 14 Hasil drop table 31
Gambar 3. 15 Hasil backup dengan cmd 31
Gambar 3. 16 Akses phpMyAdmin 32
Gambar 3. 17 Database yang akan dibackup di phpMyAdmin 32
Gambar 3. 18 Tab export phpMyAdmin 32
Gambar 3. 19 File sql dengan phpMyAdmin 33
Gambar 3. 20 Akses phpMyAdmin untuk restore 34
Gambar 3. 21 Nama database yang akan menjadi tempat restore 34
Gambar 3. 22 Tab import phpMyAdmin 34
Gambar 3. 23 Choose file phpMyAdmin 35
Gambar 3. 24 Format import phpMyAdmin 35
Gambar 3. 25 Hasil restore dengan phpMyAdmin 35
Gambar 3. 26 Hasil insert kasir 36

Gambar 3. 27 Hasil insert konsumen 37
Gambar 3. 28 Hasil insert item 38
Gambar 3. 29 Hasil insert transaksi 39
Gambar 3. 30 Hasil insert detail_transaksi 40
Gambar 3. 31 Hasil select semua kolom 40
Gambar 3. 32 Hasil select kolom spesifik 41
Gambar 3. 33 Hasil select dengan kondisi tertentu 42
Gambar 3. 34 Hasil select join 42
Gambar 3. 35 Hasil update 43
Gambar 3. 36 Sebelum delete 43
Gambar 3. 37 Sesudah delete 44
Gambar 3. 38 Hasil urutan data 44
Gambar 3. 39 Hasil agregat 44
Gambar 3. 40 Hasil between 45
Gambar 3. 41 Hasil like 45
Gambar 3. 42 Hasil mengganti nama field keluaran 46
Gambar 3. 43 Hasil menambahkan baris teks keluaran 47
Gambar 3. 44 Hasil current_date 47
Gambar 3. 45 Hasil current_time 48
Gambar 3. 46 Detail Pembayaran dan Kasir Transaksi 49
Gambar 3. 47 Menampilkan Daftar Unik Total Nilai Transaksi 50
Gambar 3. 48 Menampilkan Data Transaksi 50
Gambar 3. 49 Menampilkan Daftar Item dengan Harga di Atas 40.000 51
Gambar 3. 50 Menampilkan Jumlah Total Kuantitas Terjual per Item 51
Gambar 3. 51 Menampilkan Rekapitulasi Kuantitas Terjual Item di Atas 2 Unit 52
Gambar 3. 52 Menampilkan Data Transaksi Berdasarkan Total Nilai Terbesar .. 52
Gambar 3. 53 Hasil Commit 54
Gambar 3. 54 Hasil Rollback 55
Gambar 3. 55 Hasil Savepoint 56
Gambar 4. 1 Daftar Tabel dalam Basis Data 57
Gambar 4. 2 Struktur Tabel Item 57
Gambar 4. 3 Struktur Tabel Kasir 58

Gambar 4. 4 Struktur Tabel Konsumen 58
Gambar 4. 5 Struktur Tabel Transaksi 59
Gambar 4. 6 struktur detail_transaksi 59
Gambar 4. 7 index 1 60
Gambar 4. 8 index 2 60
Gambar 4. 9 index 3 61
Gambar 4. 10 index 4 61
Gambar 4. 11 index 5 61
Gambar 4. 12 index 6 62
Gambar 4. 13 login 63
Gambar 4. 14 dashboard 63
Gambar 4. 15 kasir lihat 64
Gambar 4. 16 kasir tambah 65
Gambar 4. 17 kasir ubah 65
Gambar 4. 18 kasir hapus 66
Gambar 4. 19 konsumen lihat 67
Gambar 4. 20 konsumen tambah 67
Gambar 4. 21 konsumen ubah 68
Gambar 4. 22 konsumen hapus 68
Gambar 4. 23 item lihat 69
Gambar 4. 24 item tambah 70
Gambar 4. 25 item ubah 70
Gambar 4. 26 item hapus 71
Gambar 4. 27 transaksi lihat 72
Gambar 4. 28 transaksi tambah 72
Gambar 4. 29 transaksi ubah 73
Gambar 4. 30 transaksi hapus 74
Gambar 4. 31 transaksi detail lihat 74
Gambar 4. 32 transaksi detail tambah 75
Gambar 4. 33 *transaksi detail ubah* 76
Gambar 4. 34 transaksi detail hapus 76
Gambar 4. 35 cetak 77

DAFTAR TABEL

Tabel 2 1 Analisis data 14
Tabel 2 2 Kamus data table kasir 19
Tabel 2 3 Kamus data table konsumen 19
Tabel 2 4 Kamus data table item 19
Tabel 2 5 Kamus data table transaksi 20
Tabel 2 6 Kamus data table detail_transaksi 20
Tabel 2 7 Fungsi sub menu physical data model 22

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dunia wirausaha global menunjukkan pertumbuhan yang signifikan, dengan sekitar 665 juta wirausahawan di seluruh dunia dan tingkat pertumbuhan *startup* tahunan rata-rata global yang diperkirakan mencapai 21% pada tahun 2025. Sejalan dengan dinamika ini, pasar kuliner global juga ikut menjadi pilar ekonomi, dimana pasar kuliner tumbuh pesat dengan *Compound Annual Growth Rate* (CAGR) sekitar 14-18% dalam beberapa tahun yang akan datang, hal ini tentunya menjadikan sektor kuliner sebagai bagian krusial dalam dinamika ekonomi. Setiap usaha atau bisnis tidak pernah luput dari proses transaksi. Transaksi adalah aktivitas dasar dalam kegiatan ekonomi yang melibatkan pertukaran barang, jasa, atau uang. Transaksi ini bukan hanya berfungsi sebagai sarana pertukaran barang atau jasa, tetapi juga sebagai elemen krusial dalam pencatatan keuangan dan indikator pertumbuhan ekonomi suatu entitas Oleh karena itu, setiap detail dari transaksi tersebut harus dicatat secara formal dan akurat melalui sebuah dokumen penting.

Keakuratan dan detail dari setiap transaksi selanjutnya akan dicatat secara formal melalui dokumen yang disebut struk transaksi atau bukti pembayaran. Struk ini ialah sumber data primer yang mencakup informasi penting seperti tanggal, waktu, rincian item yang dibeli, hingga total pembayaran. Namun, di tengah peningkatan jumlah usaha dan intensitas transaksi di era digital ini, data yang tercatat pada struk sering kali menjadi tantangan. Pengelolaan data dalam bentuk fisik atau manual ditulis tidak lagi efisien untuk mendukung analisis suatu bisnis. Hal ini dikarenakan proses manual sangat rentan dan berisiko terhadap kesalahan manusia dalam pencatatan, dapat berpotensi menimbulkan redudansi data (duplikasi informasi), serta menyulitkan proses pencarian, agregasi, dan pelaporan suatu data. Akibatnya, usaha kuliner menghadapi risiko ketidakakuratan laporan keuangan, kerugian finansial karena selisih kas yang tidak terlacak,

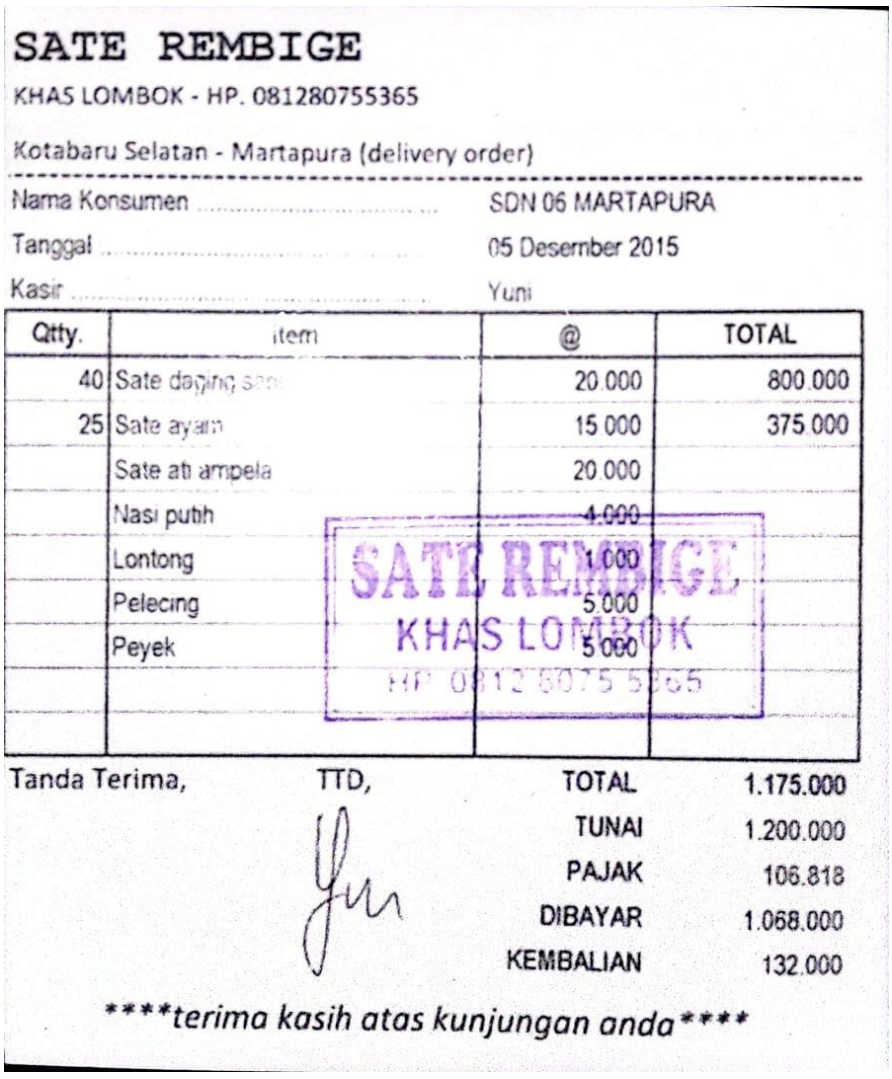
serta kelemahan dalam pengambilan keputusan strategis karena minimnya data yang dapat diandalkan. Oleh karena itu, diperlukan transformasi data struk menjadi format yang terstruktur dan siap olah. Dalam konteks ini, Struk Transaksi Sate Rembige dipilih sebagai objek studi kasus. Struk ini merepresentasikan data nyata dari usaha kuliner yang padat transaksi, yang memerlukan permodelan dan pengelolaan sistematis. Untuk mengatasi kompleksitas dan risiko tersebut, diperlukan sistem terpusat yang mampu menstrukturkan dan mengelola data transaksi, dan inilah mengapa Sistem Manajemen Basis Data menjadi solusi fundamental.

Untuk mengatasi tantangan pengelolaan dan analisis data transaksional tersebut, Sistem Manajemen Basis Data (DBMS) menjadi solusi fundamental. Basis data adalah kumpulan data yang terorganisir dan saling berhubungan, dikelola sedemikian rupa sehingga mudah diakses, dimanipulasi, dan diperbarui. Fungsi utamanya mencakup mengelompokkan data, menghindari duplikasi, mempermudah akses dan pembaruan, serta menjaga integritas dan kualitas data. Dalam makalah ini, MySQL dipilih sebagai *Relational Database Management System* (RDBMS) yang digunakan, mengingat popularitas, fleksibilitas, dan kapabilitasnya dalam menangani data terstruktur. Melalui analisis Struk Transaksi Sate Rembige, data akan dimodelkan dan diimplementasikan secara praktis menggunakan MySQL. Penerapan ini memungkinkan eksekusi berbagai perintah *Structured Query Language* (SQL), baik untuk definisi data (DDL) maupun manipulasi data (DML). Dengan demikian, tujuan utama makalah ini adalah mendemonstrasikan perancangan basis data transaksi yang efektif dan fungsional dari sebuah struk sederhana.

1.2 *Form* Transaksional

Sate Rembige merupakan sebuah unit usaha kuliner yang terkenal dengan keahliannya menyajikan hidangan sate khas Lombok. Berdasarkan dokumen transaksi yang menjadi objek studi, lokasi usaha ini tercatat di Kotabaru Selatan, Martapura. Analisis ini dilakukan pada data transaksi yang terekam pada 05 Desember 2015, yang diproses oleh kasir Yuni, dan melibatkan konsumen SDN 06 MARTAPURA. Struk tersebut mencakup

berbagai elemen penting yang dapat dikelompokkan menjadi data transaksi dan data item. Data item merinci pembelian dan sate ayam. Sementara itu, data transaksi mencakup total kotor sebelum perhitungan akhir, nilai pajak, tunai, jumlah dibayar, dan kembalian. Semua elemen tersebut dapat direpresentasikan secara terstruktur dalam model relasional menggunakan tabel-tabel seperti kasir, konsumen, item, transaksi, dan detail_transaksi, elemen – elemen tersebut menunjukkan bagaimana data dari berbagai entitas saling terhubung untuk menghasilkan informasi transaksi yang lengkap. Berikut adalah visualisasi dari struk transaksi Sate Rembige yang dijadikan objek analisis dalam makalah ini :



SATE REMBIGE

KHAS LOMBOK - HP. 081280755365

Kotabaru Selatan - Martapura (delivery order)

Nama Konsumen ..... SDN 06 MARTAPURA

Tanggal ..... 05 Desember 2015

Kasir ..... Yuni

Qtty.	item	@	TOTAL
40	Sate daging sapi	20.000	800.000
25	Sate ayam	15.000	375.000
	Sate ati ampela	20.000	
	Nasi putih	4.000	
	Lontong	1.000	
	Pelecing	5.000	
	Peyek	5.000	

SATE REMBIGE KHAS LOMBOK HP 0812 8075 5365

Tanda Terima, TTD,

TOTAL	1.175.000
TUNAI	1.200.000
PAJAK	106.818
DIBAYAR	1.068.000
KEMBALIAN	132.000

****terima kasih atas kunjungan anda****

Gambar 1. 1 Form transaksional sate rembige

BAB II
DESAIN BASIS DATA

2.1 *Entity Relationship* Diagram (ERD)

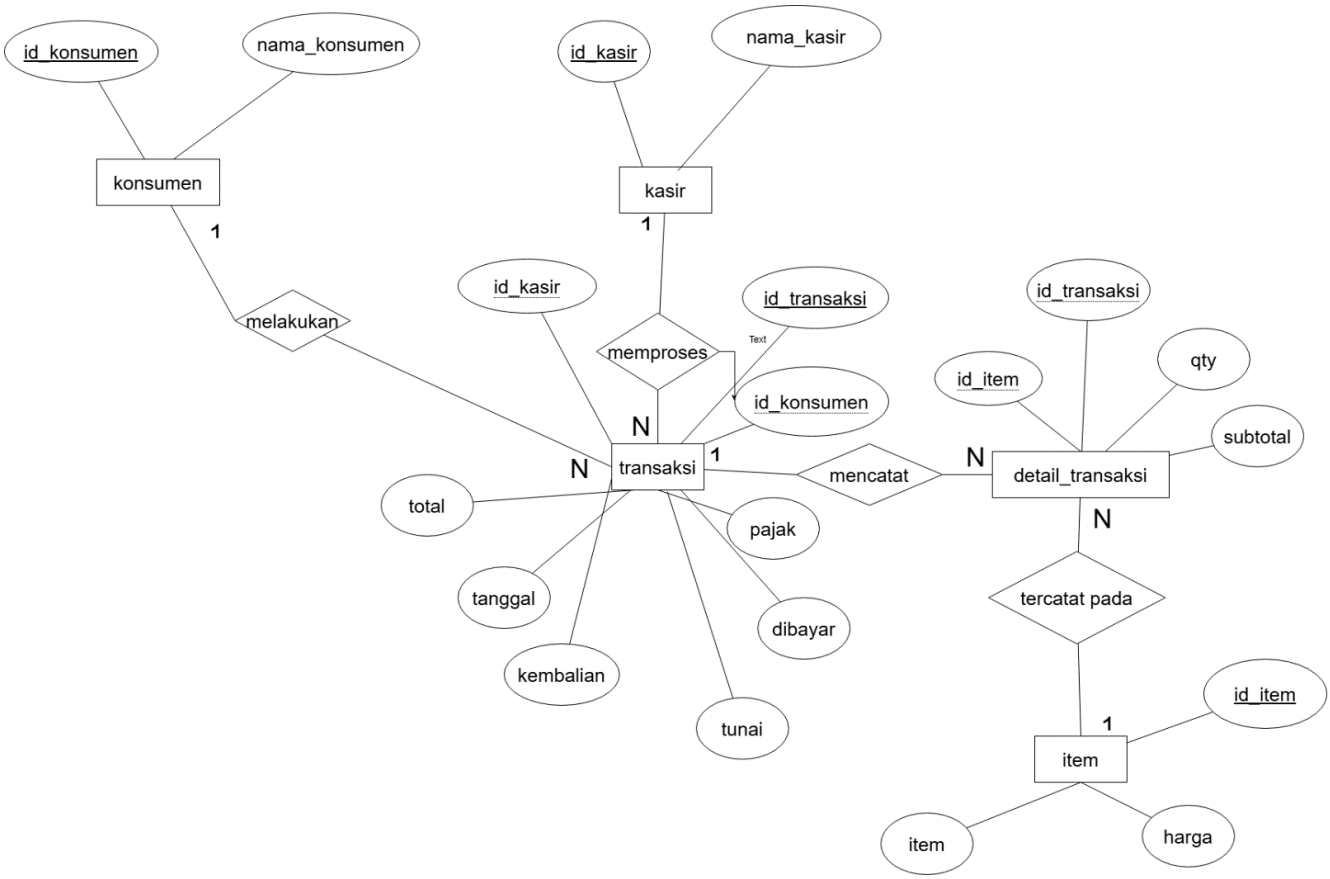
Entity Relationship Diagram (ERD) adalah langkah pertama dan paling mendasar dalam merancang basis data relasional. Konsep ini sangat penting karena berfungsi sebagai peta visual yang menggambarkan struktur data secara sederhana. Menurut para ahli, ERD dapat dipandang sebagai model jaringan abstrak yang menyusun data dalam sistem. Selain itu, ERD juga merupakan model yang berisi semua elemen penting yaitu entitas dan relasi. Entitas adalah suatu elemen yang nantinya akan disimpan dalam *database* tersebut, sedangkan relasi adalah hubungan antar entitas lengkap dengan atribut yang mewakili fakta nyata di lapangan.

ERD adalah alat komunikasi visual tingkat tinggi yang digunakan oleh profesional sistem untuk mendefinisikan kebutuhan data bisnis, menjelaskan bagaimana data-data tersebut berelasi satu sama lain, dan menentukan hak akses data. Adapun manfaat utama ERD sebagai berikut :

1. memvisualisasi struktur data untuk memberikan gambaran yang jelas dan ringkas mengenai keseluruhan struktur basis data.
2. meningkatkan ketergantungan perancangan
3. memudahkan pemahaman dan komunikasi
4. perencanaan dan evaluasi *database*
5. dokumentasi system untuk mempermudah pemeliharaan dan pengembangan dimasa depan
6. alat analisis untuk mengidentifikasi pola dan ketergantungan data yang mungkin sulit dilihat

Dalam menganalisis kasus penjualan Sate Rembige, kebutuhan akan ERD menjadi sangat penting karena data transaksi yang awalnya hanya tercatat pada dokumen fisik (struk). ERD berfungsi untuk memecah data

non-terstruktur dari struk tersebut menjadi entitas-entitas logis yang terpisah, lalu mendefinisikan aturan hubungan antar. Dengan demikian, kerangka ERD yang dihasilkan menjadi panduan yang valid dan terstruktur sebelum diimplementasikan ke dalam *database* MySQL. Berikut ini *Entity Relationship Diagram* (ERD) dari hasil analisis struk transaksi Sate Rembige:



Gambar 2. 1 ERD

Desain ERD ini melibatkan lima entitas utama yaitu kasir, konsumen, item, transaksi, dan detail_transaksi. Model data ini diatur oleh relasi satu ke banyak (1:N), yang mencerminkan alur transaksi. Entitas kasir dan konsumen memiliki hubungan 1:N dengan Transaksi, yang berarti satu kasir dapat memproses banyak Transaksi, dan satu Konsumen dapat melakukan banyak Transaksi. Entitas Transaksi merepresentasikan data seperti tanggal dan total pembayaran yang terhubung ke detail_transaksi (1:N), yang menampung rincian pembelian per baris. Detail transaksi sendiri merupakan entitas yang mengambil *Foreign Key* dari Transaksi dan

Item, menjadikannya entitas penghubung dengan *Primary Key* ganda, serta menunjukkan bahwa satu jenis item dapat muncul di banyak detail transaksi. Keseluruhan model ini memastikan setiap elemen pada struk terwakili secara logis tanpa adanya duplikasi data.

Dari *Entity Relationship Diagram* diatas, berikut daftar lengkap analisis data yang dibutuhkan :

No.	Nama Tabel	Atribut
1	Kasir : Mencatat identitas dasar petugas yang memproses penjualan	• **Id_kasir: *Primary Key* (PK).** Kode unik identifikasi petugas kasir. • Nama_kasir: nama lengkap petugas kasir yang bertugas.
2	Konsumen : Mencatat identitas dasar pelanggan yang melakukan pembelian.	• **id_konsumen: *Primary Key* (PK)**. Kode unik identifikasi pelanggan. • Nama_konsumen: nama pelanggan atau instansi yang melakukan pembelian
3	Item : Mencatat detail produk atau menuyang dijual beserta harga dasarnya.	• **Id_item: *Primary Key (PK)***. Kode unik identifikasi produk. • Item: nama produk menu. • Harga: harga satuan dari item tersebut.
4	Transaksi : Mencatat data utama transaksi secara keseluruhan.	• **Id_transaksi: *Primary Key* (PK)**. Kode unik untuk satu nota/transaksi.

		• id_kasir: *foreign key* (FK) ke tabel Kasir. Menunjukkan petugas yang memproses. • id_konsumen: *foreign key* (FK) ke tabel Konsumen. Menunjukkan pelanggan yang membeli. • Tanggal: tanggal dan waktu transaksi dilakukan. • Pajak: nominal pajak yang dikenakan pada transaksi tersebut. • Total: total biaya semua item sebelum pajak dihitung. • Dibayar: nominal uang yang dibayarkan oleh pelanggan. • Kembalian: jumlah kembalian yang diterima pelanggan • Tunai: total nominal yang harus dibayar konsumen (termasuk pajak).
5	Detail_transaksi : Mencatat rincian baris item yang dibeli, berfungsi memecah relasi Many to Many konseptual.	• id_transaksi: foreign key ke tabel transaksi • id_item: foreign key ke tabel item • qty: jumlah item yang dibeli • subtotal: Total harga untuk baris item.

Tabel 2 1 Analisis data

2.2 *Logical Record Structure* (LRS)

LRS adalah representasi dari struktur record-record pada tabel-tabel yang terbentuk dari hasil relasi antar himpunan entitas. LRS berperan sebagai jembatan penting antara model konseptual (ERD) ke model fisik. LRS berfungsi untuk menentukan kardinalitas, jumlah tabel, dan yang

paling krusial, penempatan *Foreign Key* (FK) yang akan mendefinisikan relasi.

Dalam menganalisis kasus penjualan Sate Rembige, LRS ini sangat dibutuhkan. Setelah ERD mengidentifikasi Entitas, LRS yang akan bertugas secara spesifik menentukan dimana kolom *Foreign Key* harus ditempatkan. Misalnya, LRS menentukan penempatan kunci asing (id_kasir dan id_konsumen) di tabel Transaksi, serta bagaimana relasi konseptual antara transaksi dan item dipecah menjadi entitas perantara (detail_transaksi) untuk menghasilkan relasi *One to Many* yang tepat. Proses ini memastikan tidak ada data yang hilang atau tumpang tindih saat struktur dikonversi ke *database* relasional.

Kardinalitas adalah konsep penting dalam basis data yang fungsinya untuk menunjukan jumlah kemunculan terbanyak yang diizinkan dalam suatu relasi dan mengacu pada jumlah nilai unik dalam kolom tabel relasional relatif terhadap jumlah total baris dalam tabel. Ada 3 jenis kardinalitas yang ada didalam bagan LRS, yaitu :

1. *One to One*

   Relasi *One to One* berarti bahwa di dalam suatu hubungan, setiap anggota dari satu kelompok data (Entitas A) hanya boleh terhubung dengan paling banyak satu anggota dari kelompok data lainnya (Entitas B). Demikian pula sebaliknya, setiap anggota dari Entitas B juga hanya boleh terhubung dengan paling banyak satu anggota dari Entitas A. Hubungan ini bersifat unik dan eksklusif di kedua arah.

2. *One to Many*

   One to Many (Satu ke Banyak) menunjukkan hubungan yang tidak simetris: satu anggota dari Entitas A dapat terhubung dengan banyak anggota dari Entitas B, namun setiap anggota di Entitas B secara ketat hanya boleh terhubung dengan paling banyak satu anggota dari Entitas A. Sebagai contoh, dalam sistem penjualan, satu Transaksi dapat mencakup banyak *item* (One to Many), sedangkan setiap baris *item* hanya tercatat pada satu Transaksi tertentu.

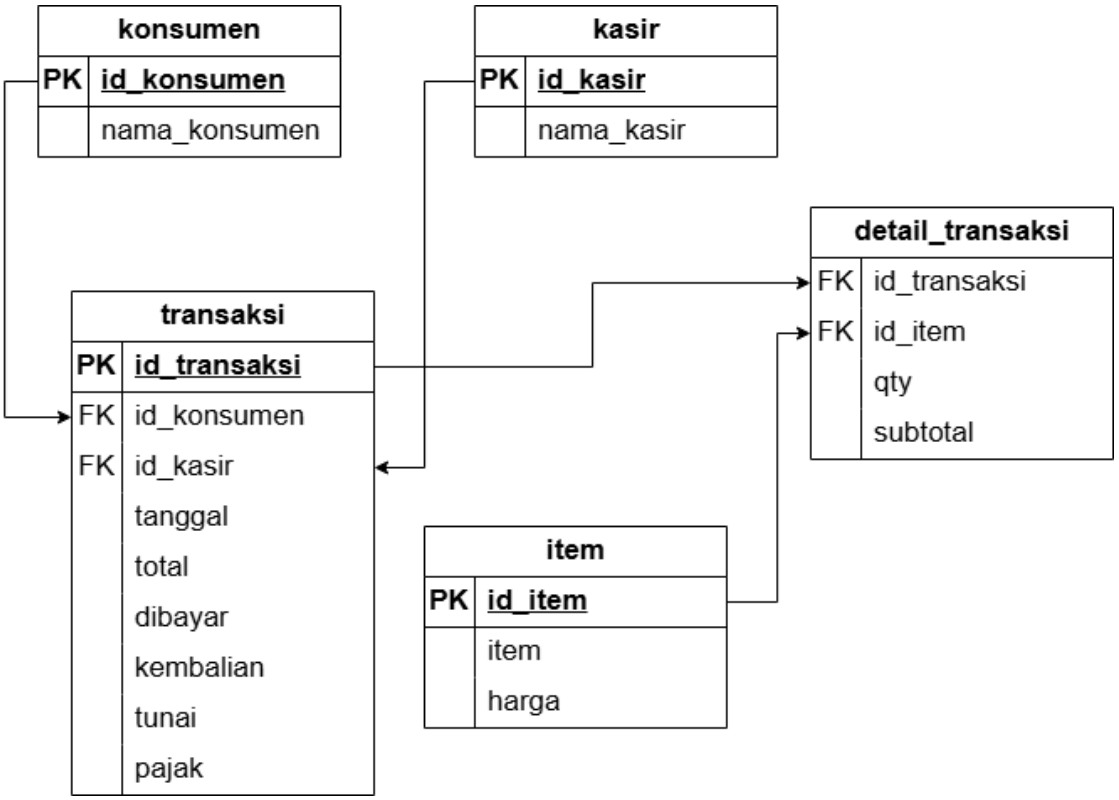
3. *Many to Many*

   Relasi *Many to Many* adalah hubungan paling fleksibel di mana setiap anggota dari kelompok data pertama (Entitas A) dapat terhubung dengan banyak anggota dari kelompok data kedua (Entitas B), dan kondisi ini berlaku juga sebaliknya. Setiap anggota dari Entitas B juga dapat terhubung dengan banyak anggota dari Entitas A. Hubungan ini seringkali memerlukan pembuatan tabel perantara dalam basis data untuk merekam semua keterkaitan yang mungkin terjadi.

Berdasarkan hasil konversi dari ERD, model LRS *Transaksi Sate Rembige* didominasi oleh relasi One to Many (1:N). Relasi ini ditunjukkan oleh:

1. Kasir ke Transaksi (1:N): satu kasir dapat memproses banyak transaksi. (id_kasir sebagai FK di Transaksi).
2. Konsumen ke transaksi (1:N): satu konsumen dapat melakukan banyak Transaksi. (id_konsumen sebagai FK di Transaksi).
3. Transaksi ke Detail_Transaksi (1:N): satu transaksi terdiri dari banyak Detail_Transaksi. (id_transaksi sebagai FK/PK di Detail_Transaksi).
4. Item ke Detail_Transaksi (1:N): satu item dapat muncul di banyak Detail_Transaksi. (id_item sebagai FK/PK di Detail_Transaksi).

Berikut ini adalah *Logical Record Structure* (LRS) dari Transaksi *Sate Rembige*:



Gambar 2. 2 LRS

Berdasarkan diagram LRS di atas, relasi tersebut akan membentuk 5 tabel utama dalam basis data. Notasi ini secara eksplisit mengidentifikasi *Primary Key* (PK) dan *Foreign Key* (FK).

1. Tabel Master

   Tabel ini menyimpan data referensi yang relatif statis dan menjadi acuan utama (sisi *'One'* / 1) dalam setiap relasi dengan tabel transaksi.

   1. Kasir (id_kasir, nama_kasir, TTD)

      Kunci utama pada tabel kasir terletak pada id_kasir

   2. Konsumen (id_konsumen, nama_konsumen)

      Kunci utama pada tabel konsumen terletak pada id_konsumen.

   3. Item (id_item, item, harga)

      Kunci utama pada tabel item terletak pada id_tem

2. Transaksi (Data Peristiwa)

Tabel ini menyimpan data yang bersifat dinamis, mencatat peristiwa bisnis, dan mendefinisikan hubungan (relasi) dengan Tabel Master melalui *Foreign Key*.

4. Transaksi (id_transaksi, id_kasir, id_konsumen, tanggal, total, pajak, dibayar, kembalian, tunai)

   Kunci utama pada tabel transaksi terletak pada id_transaksi, sedangkan foreign key pada tabel transaksi adalah id_kasir dan id_konsumen yang mendefinisikan relasi 1:N dari tabel master

5. Detail_Transaksi (id_transaksi, id_item, qty, subtotal).

   Tabel perantara/rincian transaksi dengan Kunci Gabungan (id_transaksi dan id_item), yang keduanya berfungsi sebagai Foreign Key.

2.3 *Physical Data Model*

Physical Data Model merupakan visualisasi dari *database* yang akan dibuat dengan mempertimbangkan DBMS mana yang akan digunakan. Model ini merepresentasikan struktur tabel yang terperinci, di mana setiap komponen logis diterjemahkan menjadi objek fisik, seperti tabel, *view*, dan referensi. *Physical Data Model* berfungsi sebagai *blueprint* yang mendefinisikan tabel yang terstruktur, termasuk nama kolom, tipe data kolom, *primary key*, *foreign key* dan *relationship* yang menghubungkan satu tabel dengan tabel lainnya.

A. Kamus data

Dalam implementasi sistem transaksi *Sate Rembige*, DBMS yang akan digunakan adalah MySQL. Berikut adalah rincian spesifikasi untuk kelima tabel, dengan penetapan tipe data, ukuran, dan batasan (*Constraints*) yang sesuai untuk implementasi di MySQL:

1. Tabel kasir

No	Nama *Field*	Ttype/lebar	Keterangan
1	Id_kasir	Int(11)	*Primary Key, AUTO_INCREMENT*
2	Nama_kasir	Varchar(100)	*NOT NULL*

Tabel 2 2 Kamus data table kasir

2. Tabel konsumen

No	Nama *Field*	Ttype/lebar	Keterangan
1	Id_konsumen	Int(11)	*Primary Key, AUTO_INCREMENT*
2	Nama_konsumen	Varchar(100)	*NOT NULL*

Tabel 2 3 Kamus data table konsumen

3. Tabel item

No	Nama *Field*	Ttype/lebar	Keterangan
1	Id_item	Int(11)	*Primary Key, AUTO_INCREMENT*
2	Item	Varchar(100)	*NOT NULL*
3	Harga	INT(11)	*NOT NULL*

Tabel 2 4 Kamus data table item

4. Tabel transaksi

No	Nama *Field*	Ttype/lebar	Keterangan
1	Id_transaksi	Int(11)	*Primary Key, AUTO_INCREMENT*
2	Id_konsumen	Varchar(100)	*Foreign Key*
3	Id_kasir	INT(11)	*Foreign Key*
4	Tanggal	date	*NOT NULL*

5	Total	Int(11)	*NOT NULL*
6	Dibayar	Int(11)	*NOT NULL*
7	Kembalian	Int(11)	*NOT NULL*
8	Tunai	Int(11)	
10	pajak	Decimal(10,2)	

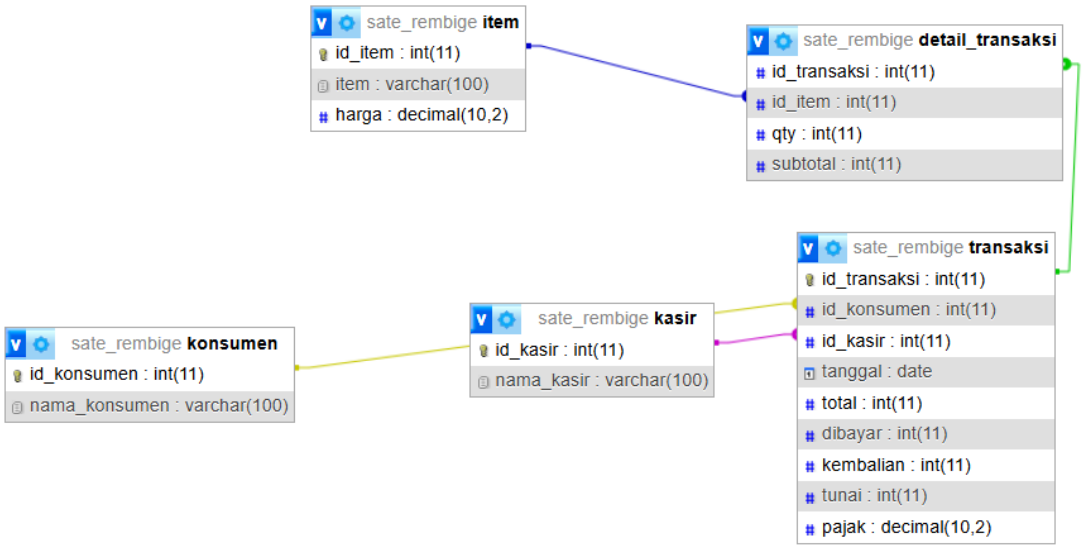
Tabel 2 5 Kamus data table transaksi

5. Detail_transaksi

No	Nama *Field*	Ttype/lebar	Keterangan
1	Id_transaksi	Int(11)	*Foreign Key*
2	Id_item	Varchar(100)	*Foreign Key*
3	Qty	Int(11)	
4	Subtotal	Int(11)	

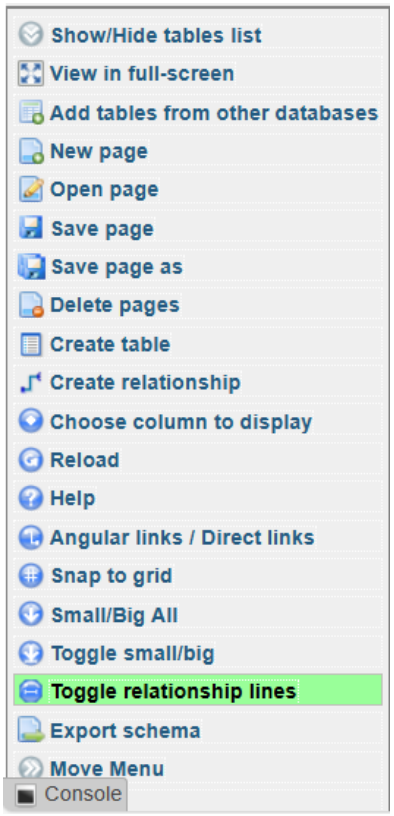
Tabel 2 6 Kamus data table detail_transaksi

B. Implementasi *Physical Data Model*



Gambar 2. 3 Physical data model

C. Fungsi Sub Menu pada *Physical Data Model*



Gambar 2. 4 Sub menu physical data model

Sub Menu	Fungsi
Show/hide tabels list	Menampilkan atau menyembunyikan daftar tabel.
View in full-screen	Mengubah tampilan ke mode layar penuh
Add tabels from other databases	Menambahkan tabel dari *database* lain ke skema yang sedang dikerjakan
New page	Membuat halaman baru untuk desain skema baru
Open page	Membuka halaman desain yang sudah ada
Save page	Menyimpan halaman desain saat ini
Save page as	Menyimpan halaman dengan nama atau versi berbeda
Delete pages	Menghapus halaman desain yang dipilih.

Create tabel	Membuat tabel baru langsung dari tampilan desain
Create relationship	Membuat relasi antar tabel
Choose column to display	Memilih kolom mana yang ingin ditampilkan pada diagram
reload	Memuat ulang tampilan/desain.
Help	Membuka menu bantua
Angular links/direct links	Mengatur tampilan garis relasi (sudut atau lurus).
Snap to grid	Menyelaraskan objek ke grid agar rapi.
Small/big all	Mengecilkan atau membesarkan tampilan seluruh objek.
Toggle small/big	Mengubah ukuran objek individual.
Toggle relationship lines	Menampilkan atau menyembunyikan garis relasi
Export shema	Mengekspor skema/desain ke format eksternal.
Move menu	Memindahkan menu ke posisi lain.

Tabel 2 7 Fungsi sub menu physical data model

BAB III

STRUCTURED QUERY LANGUAGE (SQL)

3.1 *Data Definition Language* (DDL)

Data definition language (DDL) adalah sekumpulan command atau perintah dalam SQL untuk mendefinisikan skema didalam *database*, seperti membuat *database*, memodifikasi, serta menghapus struktur *database*.

3.1.1 Create Database

Untuk membuat *database* baru dapat menggunakan perintah "create database namaDatabase", untuk membuat *database* sesuai denga form transaksional sate rembige ini, *database* akan

dinamakan sate_rembige. Berikut ini adalah kode untuk create *database*:

Create database sate_rembige

Dari kode sumber DDL di atas, akan menghasilkan basis data baru dengan nama sate_rembige. Basis data ini selanjutnya akan menjadi wadah bagi semua tabel yang akan didefinisikan.

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| rs                 |
| sate_rembige       |
| test               |
+--------------------+
7 rows in set (0.001 sec)
```

Gambar 3. 1 Hasil create database

3.1.2 Show Database

Perintah show databases digunakan untuk menampilkan daftar seluruh *database* yang ada pada *server* MySQL. Berikut ini adalah kode untuk show database: ***show databases;***

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| rs                 |
| sate_rembige       |
| test               |
+--------------------+
7 rows in set (0.001 sec)
```

Gambar 3. 2 Hasil show database

Dari kode sumber DDL di atas, dihasilkan *output* berupa tabel satu kolom dengan judul *database*. Di dalam kolom tersebut, terdaftar nama-nama *database* yang sudah ada, termasuk sate_rembige, yang mengonfirmasi bahwa *database* untuk sistem transaksi Sate Rembige telah berhasil dibuat.

3.1.3 Use Database

Perintah USE namaDatabase digunakan untuk memilih dan mengaktifkan *database* tertentu agar kita dapat bekerja di dalamnya. Berikut ini adalah contoh penggunaan use database:

use database sate_rembige;

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| rs                 |
| sate_rembige       |
| test               |
+--------------------+
7 rows in set (0.001 sec)

MariaDB [(none)]> use sate_rembige;
Database changed
```

Gambar 3. 3 Hasil use database

Perintah ini menghasilkan pesan *database changed*. Pesan ini mengindikasikan bahwa aktivasi *database* sate_rembige berhasil dilakukan, dan pengguna telah beralih fokus ke *database* tersebut.

3.1.4 Create Table

Perintah *create table* digunakan untuk membuat sebuah tabel pada *database*.

- Membuat tabel kasir

Berikut ini kode untuk membuat tabel kasir :

```
MariaDB [sate_rembige]> CREATE TABLE kasir (
```

```
-> id_kasir INT PRIMARY KEY AUTO_INCREMENT,
-> nama_kasir VARCHAR(100) NOT NULL,
-> ttd VARCHAR(100)
-> );
```

Setelah kode sumber DDL tersebut dieksekusi, sistem MariaDB akan memberikan *output Query OK, 0 rows affected* (0.020 *sec*). Pesan ini mengonfirmasi bahwa perintah SQL berjalan tanpa kesalahan sintaks atau logis sehingga menghasilkan tabel bernama kasir didalam *database* sate_rembige dengan struktur id_kasir, nama_kasir dan TTD.

```
MariaDB [sate_rembige]> DESC kasir;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| id_kasir   | int(11)      | NO   | PRI | NULL    | auto_increment |
| nama_kasir | varchar(100) | NO   |     | NULL    |                |
```

Gambar 3. 4 Hasil create table kasir

- Membuat tabel konsumen

Berikut ini kode untuk membuat tabel konsumen :

```
MariaDB [sate_rembige]> CREATE TABLE konsumen (
-> id_konsumen INT PRIMARY KEY AUTO_INCREMENT,
-> nama_konsumen VARCHAR(100) NOT NULL
-> );
```

Kode diatas menghasilkan tabel bernama konsumen didalam *database* sate_rembige dengan struktur id_konsumen, nama_konsumen.

```
MariaDB [sate_rembige]> DESC konsumen;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| id_konsumen   | int(11)      | NO   | PRI | NULL    | auto_increment |
| nama_konsumen | varchar(100) | NO   |     | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
2 rows in set (0.009 sec)
```

Gambar 3. 5 Hasil create table konsumen

- Membuat tabel item

Berikut ini kode untuk membuat tabel item:

```
MariaDB [sate_rembige]> CREATE TABLE item (
    -> id_item INT PRIMARY KEY AUTO_INCREMENT,
    -> item VARCHAR(100) NOT NULL,
    -> harga INT NOT NULL
    -> );
```

Kode diatas menghasilkan tabel bernama item didalam *database* sate_rembige dengan struktur id_item, item, harga.

```
MariaDB [sate_rembige]> DESC item;
+---------+--------------+------+-----+---------+----------------+
| Field   | Type         | Null | Key | Default | Extra          |
+---------+--------------+------+-----+---------+----------------+
| id_item | int(11)      | NO   | PRI | NULL    | auto_increment |
| item    | varchar(100) | NO   |     | NULL    |                |
| harga   | int(11)      | NO   |     | NULL    |                |
+---------+--------------+------+-----+---------+----------------+
3 rows in set (0.010 sec)
```

Gambar 3. 6 Hasil create table item

- Membuat tabel transaksi

Berikut ini kode untuk membuat tabel transaksi:

```
MariaDB [sate_rembige]> CREATE TABLE transaksi(
    -> id_transaksi INT PRIMARY KEY AUTO_INCREMENT,
    -> id_konsumen INT,
    -> id_kasir INT,
    -> tanggal DATE NOT NULL,
    -> total INT NOT NULL,
    -> dibayar INT NOT NULL,
    -> kembalian INT NOT NULL,
    -> tunai INT,
    -> pajak DECIMAL(10,2),
    -> FOREIGN KEY (id_konsumen) REFERENCES
          konsumen(id_konsumen),
    -> FOREIGN KEY (id_kasir) REFERENCES kasir(id_kasir)
```

->);

Kode diatas menghasilkan tabel bernama item didalam *database* sate_rembige dengan struktur id_transaksi, id_konsumen, id_kasir, tanggal, total, dibayar, kembalian, tunai, pajak.

```
MariaDB [sate_rembige]> DESC transaksi;
+--------------+---------------+------+-----+---------+----------------+
| Field        | Type          | Null | Key | Default | Extra          |
+--------------+---------------+------+-----+---------+----------------+
| id_transaksi | int(11)       | NO   | PRI | NULL    | auto_increment |
| id_konsumen  | int(11)       | YES  | MUL | NULL    |                |
| id_kasir     | int(11)       | YES  | MUL | NULL    |                |
| tanggal      | date          | NO   |     | NULL    |                |
| total        | int(11)       | NO   |     | NULL    |                |
| dibayar      | int(11)       | NO   |     | NULL    |                |
| kembalian    | int(11)       | NO   |     | NULL    |                |
| tunai        | int(11)       | YES  |     | NULL    |                |
| pajak        | decimal(10,2) | YES  |     | NULL    |                |
```

Gambar 3. 7 Hasil create table transaksi

- Membuat tabel detail_transaksi

Berikut ini kode untuk membuat tabel transaksi:

```
MariaDB [sate_rembige]> CREATE TABLE detail_transaksi (
    -> id_transaksi INT,
    -> id_item INT,
    -> qty INT NOT NULL,
    -> subtotal INT NOT NULL,
    -> FOREIGN KEY (id_transaksi) REFERENCES
          transaksi(id_transaksi),
    -> FOREIGN KEY (id_item) REFERENCES item(id_item)
    -> );
```

Kode diatas menghasilkan tabel bernama detail_transaksi didalam *database* sate_rembige dengan struktur id_transaksi, id_item, qty, subtotal.

```
MariaDB [sate_rembige]> DESC detail_transaksi;
+--------------+---------+------+-----+---------+-------+
| Field        | Type    | Null | Key | Default | Extra |
+--------------+---------+------+-----+---------+-------+
| id_transaksi | int(11) | YES  | MUL | NULL    |       |
| id_item      | int(11) | YES  | MUL | NULL    |       |
| qty          | int(11) | NO   |     | NULL    |       |
| subtotal     | int(11) | NO   |     | NULL    |       |
+--------------+---------+------+-----+---------+-------+
4 rows in set (0.011 sec)
```

Gambar 3. 8 Hasil create table detail_transaksi

3.1.5 Alter

Alter adalah pernyataan didalam *database* mysql yang berfungsi untuk mengubah nama *field*, mengubah tipe data, menambah *field* serta menghapus *field*

3.1.5.1 Mengubah Nama *Field*

Berikut ini kode untuk mengubah nama *field*:

```
MariaDB [sate_rembige]> ALTER TABLE transaksi
    -> CHANGE COLUMN tunai jumlah_tunai
    -> INT;
```

Perintah ini secara spesifik mengubah nama kolom dari tunai menjadi jumlah_tunai, dan mengubah tipe data kolom tersebut menjadi INT (Integer). Dengan demikian, kolom tunai pada tabel Transaksi telah berhasil diperbarui namanya menjadi jumlah_tunai.

```
MariaDB [sate_rembige]> DESC transaksi;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| id_transaksi | int(11)      | NO   | PRI | NULL    | auto_increment |
| id_konsumen  | int(11)      | YES  | MUL | NULL    |                |
| id_kasir     | int(11)      | YES  | MUL | NULL    |                |
| tanggal      | date         | NO   |     | NULL    |                |
| total        | int(11)      | NO   |     | NULL    |                |
| dibayar      | int(11)      | NO   |     | NULL    |                |
| kembalian    | int(11)      | NO   |     | NULL    |                |
| jumlah_tunai | int(11)      | YES  |     | NULL    |                |
| pajak        | decimal(10,2) | YES  |     | NULL    |                |
```

Gambar 3. 9 Hasil alter mengubah nama field

3.1.5.2 Mengubah Tipe Data

Berikut ini kode untuk mengubah tipe data:

```
MariaDB [sate_rembige]> ALTER TABLE item
    -> MODIFY COLUMN harga DECIMAL(10, 2)
    -> NOT NULL;
```

Perintah ini mengubah tipe data pada *field* harga didalam tabel item dari interger menjadi decimal(10,2)

```
MariaDB [sate_rembige]> DESC item;
+---------+---------------+------+-----+---------+----------------+
| Field   | Type          | Null | Key | Default | Extra          |
+---------+---------------+------+-----+---------+----------------+
| id_item | int(11)       | NO   | PRI | NULL    | auto_increment |
| item    | varchar(100)  | NO   |     | NULL    |                |
| harga   | decimal(10,2) | NO   |     | NULL    |                |
+---------+---------------+------+-----+---------+----------------+
3 rows in set (0.001 sec)
```

Gambar 3. 10 Hasil alter mengubah tipe data

3.1.5.3 Menambah *Field*

Berikut ini kode untuk menambah *field* :

```
MariaDB [sate_rembige]> ALTER TABLE konsumen
    -> ADD COLUMN email VARCHAR(100)
    -> AFTER nama_konsumen;
```

Perintah di atas untuk menambahkan *field* email dengan tipe data varchar(100) setelah *field* nama_konsumen.

```
MariaDB [sate_rembige]> DESC konsumen;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| id_konsumen   | int(11)      | NO   | PRI | NULL    | auto_increment |
| nama_konsumen | varchar(100) | NO   |     | NULL    |                |
| email         | varchar(100) | YES  |     | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
3 rows in set (0.001 sec)
```

Gambar 3. 11 Hasil alter menambah field

3.1.5.4 Menghapus *Field*

Berikut ini kode untuk menambah *field* :

```
MariaDB [sate_rembige]> ALTER TABLE konsumen
    -> DROP COLUMN email;
```

Field yang dihapus adalah email di tabel konsumen.

```
MariaDB [sate_rembige]> DESC konsumen;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| id_konsumen   | int(11)      | NO   | PRI | NULL    | auto_increment |
| nama_konsumen | varchar(100) | NO   |     | NULL    |                |
+---------------+--------------+------+-----+---------+----------------+
2 rows in set (0.001 sec)
```

Gambar 3. 12 Hasil alter menghapus field

3.1.6 Drop

Drop adalah perintah untuk menghapus suatu *database*/tabel.

A. drop database

drop database adalah perintah untuk menghapus *database*

Drop database sate_rembige;

B. drop tabel

drop tabel adalah perintah untuk menghapus tabel di suatu *database*. Berikut ini adalah daftar seluruh tabel yang ada di database sate_rembige :

```
MariaDB [sate_rembige]> show tables;
+---------------------------+
| Tables_in_sate_rembige    |
+---------------------------+
| detail_transaksi          |
| item                      |
| kasir                     |
| konsumen                  |
| transaksi                 |
+---------------------------+
5 rows in set (0.001 sec)
```

Gambar 3. 13 Seluruh table di database sate_rembige

Berikut ini adalah perintah untuk menghapus tabel :

Drop table detail_transaksi;

Tabel yang dihapus adalah detail_transaksi.

```
+------------------------+
| Tables_in_sate_rembige |
+------------------------+
| item                   |
| kasir                  |
| konsumen               |
| transaksi              |
+------------------------+
4 rows in set (0.001 sec)
```

Gambar 3. 14 Hasil drop table

3.1.7 Backup Database

Backup database adalah salah satu operasi pada SQL untuk membuat salinan file SQL yang berisi seluruh struktur dari database. *Backup* penting untuk mitigasi risiko kehilangan data akibat kegagalan sistem atau kesalan. Berikut ini kode untuk *backup database* :

A. Menggunakan cmd

Mysql -u root -p sate_rembige > C:\Users\hana\Downloads\sateremige_backup.sql

Setelah perintah dieksekusi, dihasilkan satu *file* teks dengan ekstensi **.**sql. *File* ini berisi seluruh *query* yang diperlukan untuk membangun kembali *database* sate_rembige secara utuh, termasuk *create table* dan *insert into* untuk semua data transaksi.

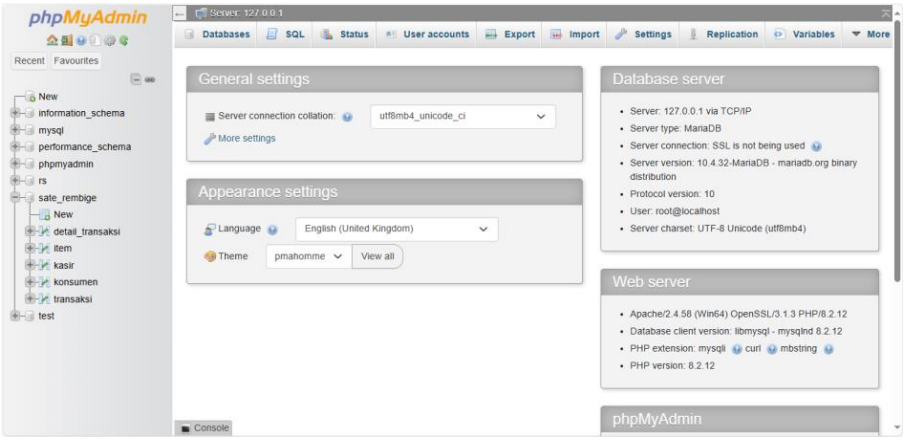


Gambar 3. 15 Hasil backup dengan cmd

B. Menggunakan phpMyAdmin

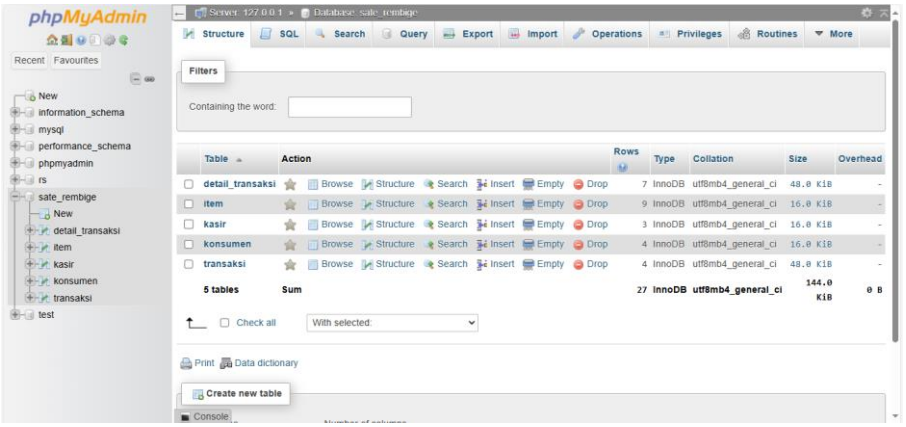
1. Buka xampp control panel
2. Tekan start pada apache dan mysql

3. Akses phpMyAdmin di browser



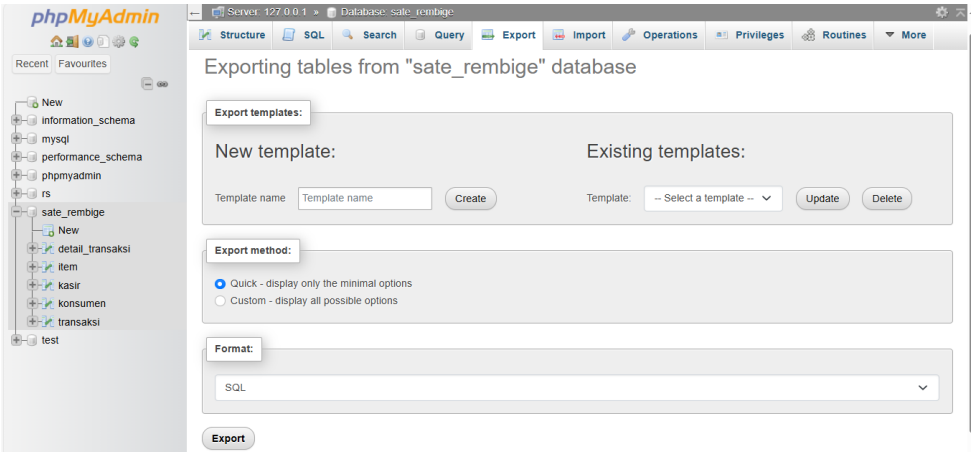
Gambar 3. 16 Akses phpMyAdmin

4. Klik nama *database* yang ingin dibackup



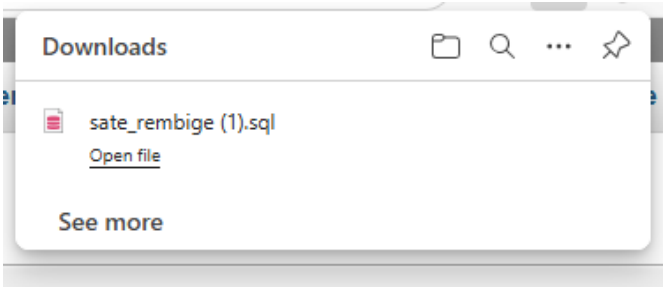
Gambar 3. 17 Database yang akan dibackup di phpMyAdmin

5. Pilih tab *export*



Gambar 3. 18 Tab export phpMyAdmin

6. Pilih metode *export* (disini saya menggunakan metode *quick*/cepat)
7. Pastikan format yang dipilih adalah SQL
8. Tekan tombol kirim, *go* atau *export*
9. Browser akan mengunduh file sql



Gambar 3. 19 File sql dengan phpMyAdmin

3.1.8 Restore Database

Restore database adalah operasi untuk mengembalikan *database* ke keadaan sebelumnya menggunakan *file backup* yang telah dibuat. Berikut ini kode untuk *restore database* :

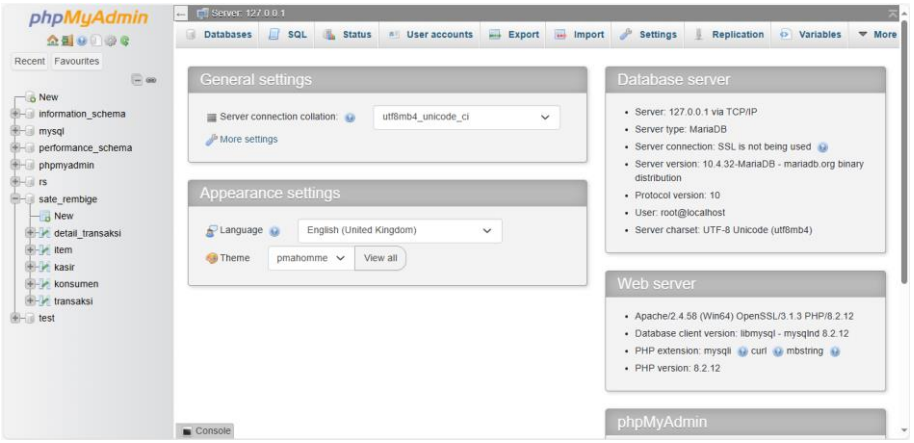
A. Menggunakan CMD

Mysql -u root -p sate_rembige <
C:\Users\hana\Downloads\sateremige_backup.sql

Seluruh struktur dan data yang terdapat dalam *file* sate_rembige_backup.sql berhasil dimasukkan kembali ke *database* sate_rembige. Hasilnya, *database* akan kembali ke kondisi saat *backup* tersebut dibuat, memastikan integritas dan ketersediaan data transaksi Sate Rembige.

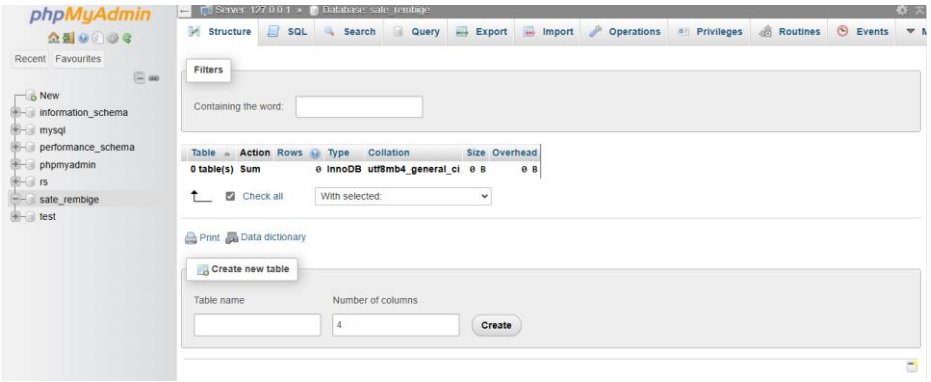
B. Menggunakan phpMyAdmin

1. Buka xampp control panel
2. Tekan start pada apache dan mysql
3. Akses phpMyAdmin di browser



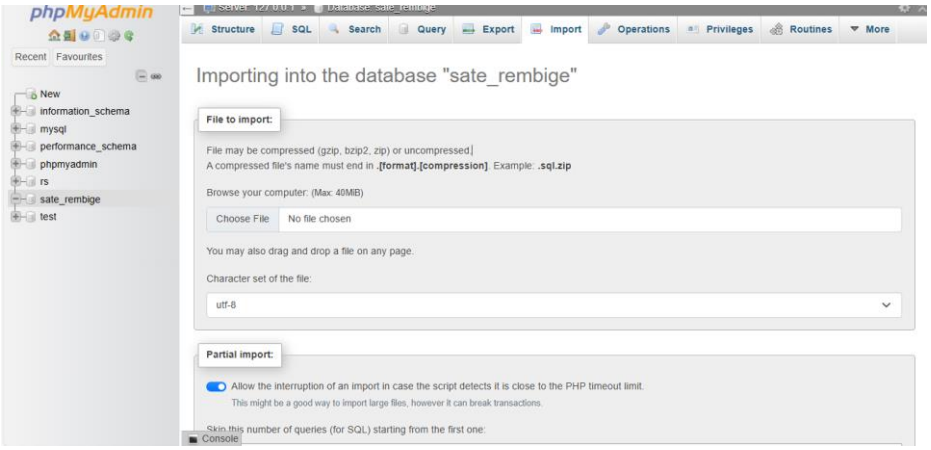
Gambar 3. 20 Akses phpMyAdmin untuk restore

4. Klik nama *database* yang ingin direstore



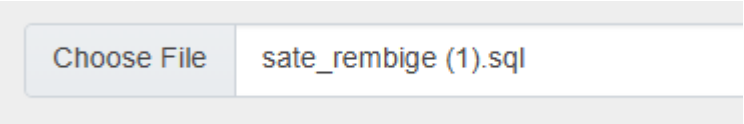
Gambar 3. 21 Nama database yang akan menjadi tempat restore

5. Pilih tab *import*



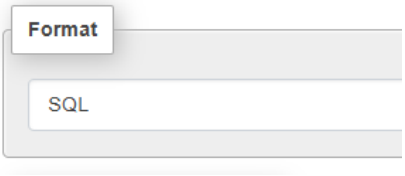
Gambar 3. 22 Tab import phpMyAdmin

6. Klik tombol *choose file* dan pilih file SQL dari hasil *backup* sebelumnya



Gambar 3. 23 Choose file phpMyAdmin

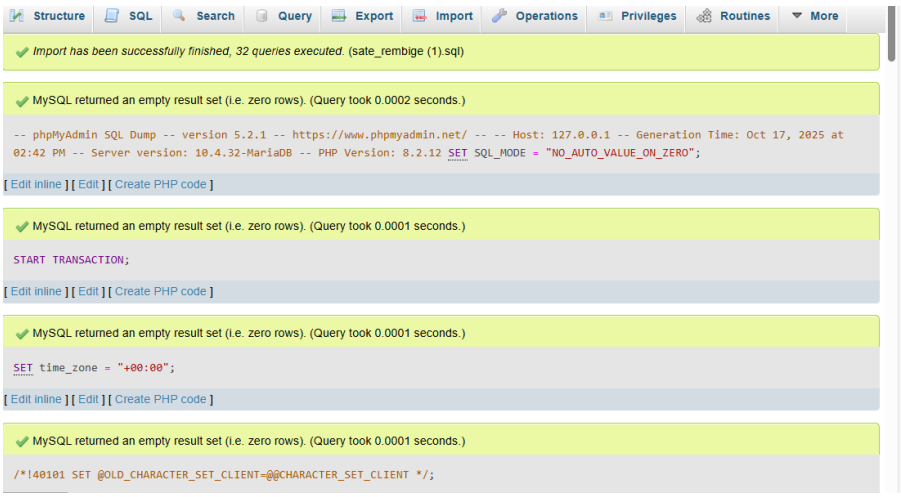
7. Pastikan format *impor* adalah SQL



Gambar 3. 24 Format import phpMyAdmin

8. Klik tombol kirim, *go* atau *import*

9. phpMyAdmin akan menjalankan semua perintah DDL dan DML dalam *file* .sql tersebut. *Database* sate_rembige akan terisi kembali dengan seluruh struktur tabel dan data transaksi yang tersimpan saat *backup* dilakukan.



Gambar 3. 25 Hasil restore dengan phpMyAdmin

3.2 *Data Manipulation Language* (DML)

Data Manipulation Language (DML) adalah sekumpulan *command* atau perintah dalam SQL yang digunakan untuk memanipulasi data di dalam objek *database* yang telah didefinisikan oleh DDL. Secara sederhana, DML berurusan dengan **isi** tabel, bukan strukturnya. Tujuan utama DML adalah mengelola informasi transaksional Sate Rembige, termasuk mencatat

penjualan, memperbarui harga menu, atau menghapus catatan kasir yang sudah tidak aktif.

3.2.1 *Insert*

Insert adalah perintah pada SQL untuk memasukkan baris data baru

A. insert kasir

kode untuk memasukkan baris data baru :

```
MariaDB [sate_rembige]> INSERT INTO kasir (
    -> nama_kasir,
    -> )
    -> VALUES
    -> ('Budi Santoso'),
    -> ('Siti Aminah'),
    -> ('Yuni');
```

Hasilnya :

```
MariaDB [sate_rembige]> SEL
+----------+--------------+
| id_kasir | nama_kasir   |
+----------+--------------+
|        1 | Budi Santoso |
|        2 | Siti Aminah  |
|        3 | Yuni         |
+----------+--------------+
3 rows in set (0.001 sec)
```

Gambar 3. 26 Hasil insert kasir

B. *insert* konsumen

Kode untuk memasukkan baris data baru :

```
MariaDB [sate_rembige]> INSERT INTO konsumen (
    -> nama_konsumen
```

->)

-> VALUES

-> ('Andi Wijaya'),

-> ('Dewi Lestari'),

-> ('Tanpa Nama'),

-> ('SDN 06 MARTAPURA');

Hasilnya :

```
MariaDB [sate_rembige]> SELECT * FROM konsumen;
+-------------+------------------+
| id_konsumen | nama_konsumen    |
+-------------+------------------+
|           1 | Andi Wijaya      |
|           2 | Dewi Lestari     |
|           3 | Tanpa Nama       |
|           4 | SDN 06 MARTAPURA |
+-------------+------------------+
4 rows in set (0.001 sec)
```

Gambar 3. 27 Hasil insert konsumen

C. *insert* item

Kode untuk memasukkan baris data baru :

MariaDB [sate_rembige]> INSERT INTO item (

-> item,

-> harga

->)

-> VALUES

-> ('Sate Rembige Ayam', 2500000),

-> ('Sate Rembige Sapi', 3000000),

-> ('Nasi Putih', 500000),

-> ('Es Jeruk', 1000000),

-> ('Sate Daging 570', 2000000),

-> ('Sate Ayam', 1500000),

-> ('Nasi Putih', 400000),

-> ('Lontong', 500000),

-> ('Pelecing Peyek', 500000);

Hasilnya :

```
MariaDB [sate_rembige]> SELECT * FROM item;
+---------+--------------------+------------+
| id_item | item               | harga      |
+---------+--------------------+------------+
|       1 | Sate Rembige Ayam | 2500000.00 |
|       2 | Sate Rembige Sapi | 3000000.00 |
|       3 | Nasi Putih         |  500000.00 |
|       4 | Es Jeruk           | 1000000.00 |
|       5 | Sate Daging 570    | 2000000.00 |
|       6 | Sate Ayam          | 1500000.00 |
|       7 | Nasi Putih         |  400000.00 |
|       8 | Lontong            |  500000.00 |
|       9 | Pelecing Peyek     |  500000.00 |
+---------+--------------------+------------+
9 rows in set (0.001 sec)
```

Gambar 3. 28 Hasil insert item

D. *insert* transaksi

Kode untuk memasukkan baris data baru :

MariaDB [sate_rembige]> INSERT INTO transaksi (

-> id_konsumen,

-> id_kasir,

-> tanggal,

-> total,

-> dibayar,

-> kembalian,

-> jumlah_tunai,

-> pajak,

->)

-> VALUES

-> (1, 1, '2025-10-15', 77000, 80000, 3000, 84000, 11000.00);

Hasilnya :

```
MariaDB [sate_rembige]> SELECT * FROM transaksi;
+--------------+-------------+----------+------------+---------+---------+-----------+--------------+----------+
| id_transaksi | id_konsumen | id_kasir | tanggal    | total   | dibayar | kembalian | jumlah_tunai | pajak    |
+--------------+-------------+----------+------------+---------+---------+-----------+--------------+----------+
|            1 |           1 |        1 | 2025-10-15 |   77000 |   80000 |      3000 |        84000 | 11000.00 |
```

Gambar 3. 29 Hasil insert transaksi

E. *insert* detail_transaksi

Kode untuk memasukkan baris data baru :

MariaDB [sate_rembige]> INSERT INTO detail_transaksi (

-> id_transaksi,

-> id_item,

-> qty,

-> subtotal

->)

-> VALUES

-> (1, 1, 2, 50000),

-> (1, 3, 4, 20000),

-> (2, 2, 1, 30000),

-> (2, 4, 1, 10000),

-> (3, 1, 1, 25000),

-> (3, 2, 1, 30000),

-> (4, 5, 40, 800000),

-> (4, 6, 25, 375000);

Hasilnya :

```
MariaDB [sate_rembige]> SELECT * FROM detail_transaksi;
+--------------+---------+-----+----------+
| id_transaksi | id_item | qty | subtotal |
+--------------+---------+-----+----------+
|            1 |       1 |   2 |    50000 |
|            1 |       3 |   4 |    20000 |
|            2 |       2 |   1 |    30000 |
|            2 |       4 |   1 |    10000 |
|            3 |       1 |   1 |    25000 |
|            3 |       2 |   1 |    30000 |
|            4 |       5 |  40 |   800000 |
|            4 |       6 |  25 |   375000 |
+--------------+---------+-----+----------+
8 rows in set (0.001 sec)
```

Gambar 3. 30 Hasil insert detail_transaksi

3.2.2 *Select*

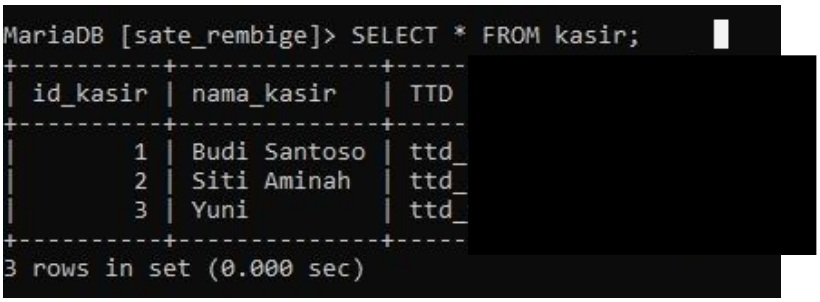
Select adalah perintah SQL yang digunakan untuk mengambil/menampilkan data dari satu atau lebih tabel didalam *database*.

A. Select untuk mengambil semua kolom

Perintah ini menggunakan simbol *asterisk* (*) setelah *SELECT*, yang berarti sistem harus mengambil dan menampilkan seluruh kolom dan seluruh baris data yang terdapat pada tabel yang disebutkan. Contoh select untuk mengambil semua kolom dari tabel kasir :

MariaDB [sate_rembige]> SELECT * FROM kasir;

Hasilnya :



Gambar 3. 31 Hasil select semua kolom

B. *Select* untuk mengambil kolom spesifik

Dalam kasus ini, hanya nama-nama kolom tertentu yang ingin dilihat yang disebutkan setelah perintah *SELECT*. Ini dilakukan untuk efisiensi dan hanya fokus pada data yang relevan. Contoh *select* untuk mengambil kolom item dan harga dari tabel item :

MariaDB [sate_rembige]> SELECT item, harga FROM Item;

Hasilnya :

```
MariaDB [sate_rembige]> SELECT item, harga FROM Item;
+--------------------+------------+
| item               | harga      |
+--------------------+------------+
| Sate Rembige Ayam  | 2500000.00 |
| Sate Rembige Sapi  | 3000000.00 |
| Nasi Putih         |  500000.00 |
| Es Jeruk           | 1000000.00 |
| Sate daging 570    | 2000000.00 |
| Sate ayam          | 1500000.00 |
| Nasi putih         |  400000.00 |
| Lontong            |  500000.00 |
| Pelecing Peyek     |  500000.00 |
+--------------------+------------+
9 rows in set (0.000 sec)
```

Gambar 3. 32 Hasil select kolom spesifik

C. *Select* dengan kondisi tertentu

Pengambilan data dilakukan dengan menambahkan klausa *where*. Klausa ini berfungsi sebagai *filter*, yang memastikan bahwa hanya baris-baris data yang memenuhi kriteria atau kondisi yang ditentukan saja yang akan ditampilkan. Contoh *select* dengan kondisi:

MariaDB [sate_rembige]> SELECT * FROM transaksi WHERE total > 100000;

Hasilnya :

```
+--------------+-------------+----------+------------+---------+---------+-----------+--------------+-----------+
| id_transaksi | id_konsumen | id_kasir | tanggal    | total   | dibayar | kembalian | jumlah_tunai | pajak     |
+--------------+-------------+----------+------------+---------+---------+-----------+--------------+-----------+
|            4 |           4 |        3 | 2015-12-05 | 1068000 | 1200000 |    132000 |      1174818 | 106818.00 |
+--------------+-------------+----------+------------+---------+---------+-----------+--------------+-----------+
1 row in set (0.001 sec)
```

Gambar 3. 33 Hasil select dengan kondisi tertentu

D. *Select* dengan *join*

Perintah ini digunakan untuk menggabungkan kolom-kolom dari dua atau lebih tabel yang memiliki relasi (hubungan) melalui *Foreign Key* tertentu. Perintah *JOIN* sangat penting dalam sistem relasional Sate Rembige untuk melihat data detail yang terpisah, misalnya menggabungkan data transaksi dengan nama kasir yang memprosesnya. Contoh *select* dengan *join* :

```
MariaDB [sate_rembige]> SELECT t.tanggal, ksr.nama_kasir, ksm.nama_konsumen, t.total
    -> FROM transaksi t
    -> JOIN kasir ksr ON t.id_kasir = ksr.id_kasir
    -> JOIN konsumen ksm ON t.id_konsumen = ksm.id_konsumen;
```

Hasilnya :

```
+------------+--------------+------------------+---------+
| tanggal    | nama_kasir   | nama_konsumen    | total   |
+------------+--------------+------------------+---------+
| 2025-10-15 | Budi Santoso | Andi Wijaya      |   77000 |
| 2025-10-15 | Siti Aminah  | Dewi Lestari     |   44000 |
| 2025-10-16 | Budi Santoso | Tanpa Nama       |   60500 |
| 2015-12-05 | Yuni         | SDN 06 MARTAPURA | 1068000 |
+------------+--------------+------------------+---------+
4 rows in set (0.001 sec)
```

Gambar 3. 34 Hasil select join

3.2.3 *Update*

Perintah *UPDATE* digunakan untuk memodifikasi data yang sudah ada di dalam tabel. Contoh kode *update* :

```
MariaDB [sate_rembige]> UPDATE transaksi
    -> SET pajak = 13000.00
    -> WHERE id_transaksi = 1;
```

Hasilnya :

```
+--------------+-------+----------+
| id_transaksi | total | pajak    |
+--------------+-------+----------+
|            1 | 77000 | 13000.00 |
+--------------+-------+----------+
1 row in set (0.001 sec)
```

Gambar 3. 35 Hasil update

3.2.4 *Delete*

Perintah *delete* digunakan untuk menghapus baris data dari tabel. Contoh kode *delete* detail_transaksi dimana id item adalah 6:

```
MariaDB [sate_rembige]> DELETE FROM detail_transaksi WHERE id_item = 6;
```

Hasilnya :

```
MariaDB [sate_rembige]> select * from detail_transaksi;
+--------------+---------+-----+----------+
| id_transaksi | id_item | qty | subtotal |
+--------------+---------+-----+----------+
|            1 |       1 |   2 |    50000 |
|            1 |       3 |   4 |    20000 |
|            2 |       2 |   1 |    30000 |
|            2 |       4 |   1 |    10000 |
|            3 |       1 |   1 |    25000 |
|            3 |       2 |   1 |    30000 |
|            4 |       5 |  40 |   800000 |
|            4 |       6 |  25 |   375000 |
+--------------+---------+-----+----------+
8 rows in set (0.001 sec)
```

Gambar 3. 36 Sebelum delete

```
MariaDB [sate_rembige]> SELECT * FROM detail_transaksi;
+--------------+---------+-----+----------+
| id_transaksi | id_item | qty | subtotal |
+--------------+---------+-----+----------+
|            1 |       1 |   2 |    50000 |
|            1 |       3 |   4 |    20000 |
|            2 |       2 |   1 |    30000 |
|            2 |       4 |   1 |    10000 |
|            3 |       1 |   1 |    25000 |
|            3 |       2 |   1 |    30000 |
|            4 |       5 |  40 |   800000 |
+--------------+---------+-----+----------+
7 rows in set (0.001 sec)
```

Gambar 3. 37 Sesudah delete

3.2.5 Urutan Data

Urutan data dalam SQL menggunakan klausa order by. Contoh kode urutan data sesuai dengan tanggal:

```
MariaDB [sate_rembige]> SELECT id_transaksi, tanggal, total
    -> FROM transaksi
    -> ORDER BY tanggal ASC, total DESC;
```

Hasilnya :

```
+--------------+------------+---------+
| id_transaksi | tanggal    | total   |
+--------------+------------+---------+
|            4 | 2015-12-05 | 1068000 |
|            1 | 2025-10-15 |   77000 |
|            2 | 2025-10-15 |   44000 |
|            3 | 2025-10-16 |   60500 |
+--------------+------------+---------+
4 rows in set (0.001 sec)
```

Gambar 3. 38 Hasil urutan data

3.2.6 Agregat

Fungsi Agregat digunakan untuk melakukan perhitungan pada sekumpulan baris dan menghasilkan nilai tungga. Contoh kode agregat :

```
MariaDB [sate_rembige]> SELECT
    -> SUM(jumlah_tunai) AS total_uang_tunai_diterima,
    -> COUNT(jumlah_tunai) AS jumlah_pembayaran_tunai,
    -> AVG(jumlah_tunai) AS rata_rata_pembayaran_tunai,
    -> MAX(jumlah_tunai) AS pembayaran_tunai_terbesar,
    -> MIN(jumlah_tunai) AS pembayaran_tunai_terkecil
    -> FROM transaksi;
```

Hasilnya :

```
+-------------------------+--------------------------+-----------------------------+---------------------------+---------------------------+
| total_uang_tunai_diterima | jumlah_pembayaran_tunai | rata_rata_pembayaran_tunai | pembayaran_tunai_terbesar | pembayaran_tunai_terkecil |
+-------------------------+--------------------------+-----------------------------+---------------------------+---------------------------+
|                 1372818 |                        4 |              343204.5000 |                1174818 |                     48000 |
+-------------------------+--------------------------+-----------------------------+---------------------------+---------------------------+
1 row in set (0.001 sec)
```

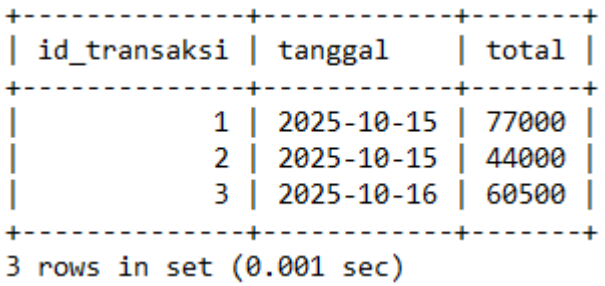
Gambar 3. 39 Hasil agregat

3.2.7 *Between*

BETWEEN digunakan untuk memfilter data dalam rentang nilai tertentu (*inklusif*). Contoh kode *between* :

```
MariaDB [sate_rembige]> SELECT id_transaksi, tanggal, total
    -> FROM transaksi
    -> WHERE total BETWEEN 40000 AND 80000;
```

Hasilnya :



```
+--------------+------------+-------+
| id_transaksi | tanggal    | total |
+--------------+------------+-------+
|            1 | 2025-10-15 | 77000 |
|            2 | 2025-10-15 | 44000 |
|            3 | 2025-10-16 | 60500 |
+--------------+------------+-------+
3 rows in set (0.001 sec)
```

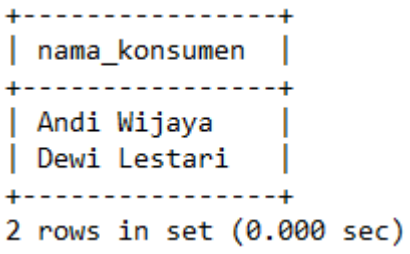
Gambar 3. 40 Hasil between

3.2.8 *Like*

Like digunakan untuk pencarian data berdasarkan pola. Contoh kode *like* :

```
MariaDB [sate_rembige]> SELECT nama_konsumen
    -> FROM konsumen
    -> WHERE nama_konsumen LIKE 'a%'
    -> OR nama_konsumen LIKE 'd%';
```

Hasilnya :



```
+----------------+
| nama_konsumen  |
+----------------+
| Andi Wijaya    |
| Dewi Lestari   |
+----------------+
2 rows in set (0.000 sec)
```

Gambar 3. 41 Hasil like

3.2.9 Mengganti Nama *Field* Keluaran

Untuk mengganti nama *field* keluaran, AS digunakan untuk memeberikan nama sementara agar lebih mudah dibaca pada kolom query. Contoh kode untuk mengganti nama *field* keluaran :

```
MariaDB [sate_rembige]> SELECT nama_kasir AS nama_pegawai,
FROM kasir;
```

Hasilnya :

```
+--------------+
| nama_pegawai |
+--------------+
| Budi Santoso |
| Siti Aminah  |
| Yuni         |
+--------------+
```

Gambar 3. 42 Hasil mengganti nama field keluaran

3.2.10 Menambahkan Baris Teks *Field* Keluaran

Untuk menampilkan kolom *output* yang berisi teks statis atau baris teks deskriptif, dapat digunakan string literal (teks yang diapit tanda kutip tunggal) yang kemudian diberi nama kolom menggunakan klausa AS. Metode ini digunakan untuk memberikan label deskriptif berulang pada setiap baris data menu Sate Rembige. Contoh kode untuk menambahkan baris teks *field* keluaran:

```
MariaDB [sate_rembige]> SELECT
   -> 'nama item:' AS keterangan,
   -> item AS menu,
   -> 'harga (rp):' AS label_harga,
   -> harga
   -> FROM item;
```

Hasilnya :

```
+------------+--------------------+-------------+------------+
| keterangan | menu               | label_harga | harga      |
+------------+--------------------+-------------+------------+
| nama item: | Sate Rembige Ayam  | harga (rp): | 2500000.00 |
| nama item: | Sate Rembige Sapi  | harga (rp): | 3000000.00 |
| nama item: | Nasi Putih         | harga (rp): |  500000.00 |
| nama item: | Es Jeruk           | harga (rp): | 1000000.00 |
| nama item: | Sate daging 570    | harga (rp): | 2000000.00 |
| nama item: | Sate ayam          | harga (rp): | 1500000.00 |
| nama item: | Nasi putih         | harga (rp): |  400000.00 |
| nama item: | Lontong            | harga (rp): |  500000.00 |
| nama item: | Pelecing Peyek     | harga (rp): |  500000.00 |
+------------+--------------------+-------------+------------+
9 rows in set (0.000 sec)
```

Gambar 3. 43 Hasil menambahkan baris teks keluaran

3.2.11 Current_Date

Fungsi current_date digunakan untuk menampilkan tanggal sistem saat *query* dijalankan. Contoh kode untuk current_date :

```
MariaDB [sate_rembige]> SELECT CURRENT_DATE() AS 'tanggal hari ini';
```

Hasilnya :

```
+------------------+
| tanggal hari ini |
+------------------+
| 2026-10-16       |
+------------------+
1 row in set (0.000 sec)
```

Gambar 3. 44 Hasil current_date

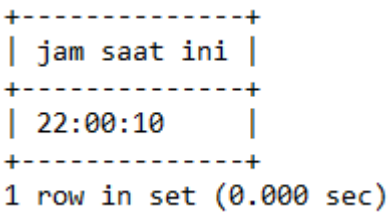
3.2.12 Current_Time

Fungsi current_time digunakan untuk menampilkan waktu sistem saat *query* dijalankan.

Contoh kode untuk current_time :

```
MariaDB [sate_rembige]> SELECT CURRENT_TIME() AS 'jam saat ini';
```

Hasilnya :



Gambar 3. 45 Hasil current_time

3.3 *Data Control Language* (DCL)

Data Control Language adalah bagian dari perintah SQL yang digunakan untuk mengatur hak akses dan keamanan pada database**.**

3.3.1 *Grant*

Memberikan hak kepada user untuk melakukan tindakan tertentu pada database.

Kode untuk membuat user baru :

```
MariaDB [sate_rembige]> CREATE USER 'namauser'@'localhost'
   -> IDENTIFIED BY 'password';
```

Kode untuk menyimpan perubahan hak akses :

FLUSH PRIVILEGES;

Kode untuk grant :

- ***GRANT SELECT ON nama_tabel TO nama_user;***
  User hanya bisa melihat data, tidak bisa mengubah.
- ***GRANT ALL PRIVILEGES ON nama_database.* TO nama_user;***
  User punya kontrol penuh atas seluruh tabel dalam database.
- ***GRANT SELECT (nama_kolom) ON nama_tabel TO nama_user;***

  User hanya boleh membaca kolom tertentu saja.

- ***GRANT CREATE, ALTER, DROP ON nama_database.* TO nama_user;***

User memiliki izin untuk mengatur struktur database.

3.3.2 *Revoke*

Mencabut izin dari user.

Contoh kode revoke :

REVOKE INSERT ON nama_tabel FROM namauser;

User *namauser* tidak lagi boleh melakukan INSERT pada *nama_tabel*.

3.4 *Data Query Language* (DQL)

Data Query Language adalah bagian dari SQL yang digunakan untuk mengambil data dari *database*. DQL Fokusnya pada membaca data, bukan mengubah atau mengelola struktur database.

3.4.1 Menampilkan Detail Pembayaran dan Kasir Transaksi

Contoh kode:

```
MariaDB [sate_rembige]> SELECT id_kasir, total, dibayar FROM transaksi;
```

Hasil :

```
+----------+---------+---------+
| id_kasir | total   | dibayar |
+----------+---------+---------+
|        1 |   77000 |   80000 |
|        2 |   44000 |   50000 |
|        1 |   60500 |  100000 |
|        3 | 1068000 | 1200000 |
+----------+---------+---------+
4 rows in set (0.000 sec)
```

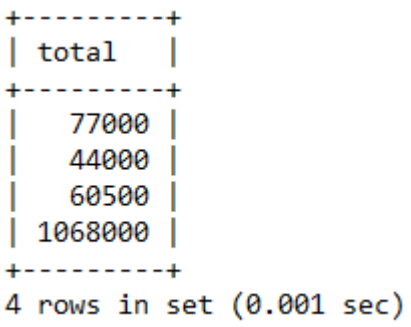
Gambar 3. 46 Detail Pembayaran dan Kasir Transaksi

3.4.2 Menampilkan Daftar Unik Total Nilai Transaksi

Contoh kode:

```
MariaDB [sate_rembige]> SELECT DISTINCT total FROM transaksi;
```

Hasil :



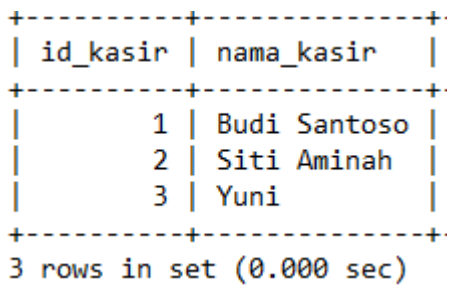
Gambar 3. 47 Menampilkan Daftar Unik Total Nilai Transaksi

3.4.3 Menampilkan Data Kasir

Memilih data dari table yang diinginkan. Contoh kode :

MariaDB [sate_rembige]> SELECT * FROM kasir;

Hasil :



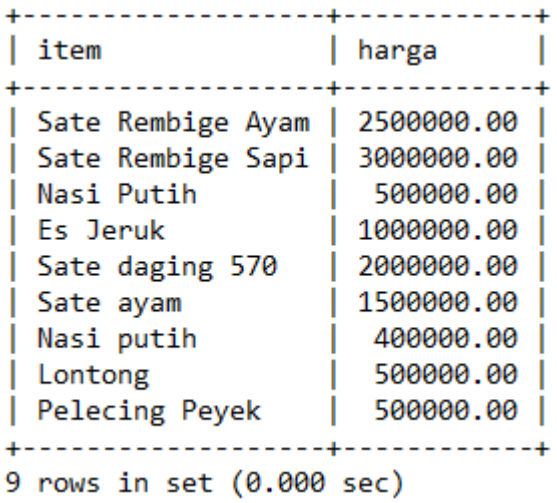
Gambar 3. 48 Menampilkan Data Kasir

3.4.4 Menampilkan Daftar Item dengan Harga di Atas 40.000

Memilih data dengan menentukan syarat data yang dipilih. Contoh kode:

MariaDB [sate_rembige]> SELECT item, harga FROM item WHERE harga > 40000;

Hasil :



item	harga
Sate Rembige Ayam	2500000.00
Sate Rembige Sapi	3000000.00
Nasi Putih	500000.00
Es Jeruk	1000000.00
Sate daging 570	2000000.00
Sate ayam	1500000.00
Nasi putih	400000.00
Lontong	500000.00
Pelecing Peyek	500000.00

9 rows in set (0.000 sec)

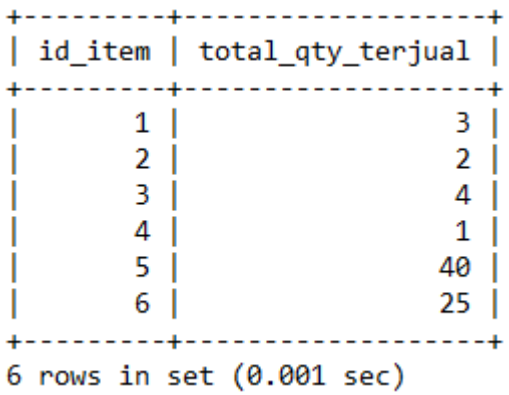
Gambar 3. 49 Menampilkan Daftar Item dengan Harga di Atas 40.000

3.4.5 Menampilkan Jumlah Total Kuantitas Terjual per Item

Mengelompokkan data dengan nilai sama. Contoh kode:

```
MariaDB [sate_rembige]> SELECT id_item, SUM(qty) AS total_qty_terjual
    -> FROM detail_transaksi
    -> GROUP BY id_item;
```

Hasil :



id_item	total_qty_terjual
1	3
2	2
3	4
4	1
5	40
6	25

6 rows in set (0.001 sec)

Gambar 3. 50 Menampilkan Jumlah Total Kuantitas Terjual per Item

3.4.6 Menampilkan Rekapitulasi Kuantitas Terjual Item > 2

Pemilihan data sesuai dengan syarat setelah pengelompokan. Contoh kode:

```
MariaDB [sate_rembige]> SELECT id_item, SUM(qty) AS total_qty_terjual
    -> FROM detail_transaksi
    -> GROUP BY id_item
    -> HAVING SUM(qty) > 2;
```

Hasil :

```
+---------+--------------------+
| id_item | total_qty_terjual |
+---------+--------------------+
|       1 |                  3 |
|       3 |                  4 |
|       5 |                 40 |
|       6 |                 25 |
+---------+--------------------+
4 rows in set (0.001 sec)
```

Gambar 3. 51 Menampilkan Rekapitulasi Kuantitas Terjual Item di Atas 2 Unit

3.4.7 Menampilkan Data Transaksi Berdasarkan Total Nilai Terbesar

Contoh kode :

```
MariaDB [sate_rembige]> SELECT * FROM transaksi ORDER BY total DESC;
```

Hasil :

```
+--------------+----------+-------------+------------+---------+---------+---------+
| id_transaksi | id_kasir | id_konsumen | tanggal    | total   | bayar   | kembali |
+--------------+----------+-------------+------------+---------+---------+---------+
|            4 |        3 |           4 | 2015-12-05 | 1068000 | 1200000 |  132000 |
|            1 |        1 |           1 | 2025-10-15 |   77000 |   80000 |    3000 |
|            3 |        1 |           3 | 2025-10-16 |   60500 |  100000 |   39500 |
|            2 |        2 |           2 | 2025-10-15 |   44000 |   50000 |    6000 |
+--------------+----------+-------------+------------+---------+---------+---------+
4 rows in set (0.001 sec)
```

Gambar 3. 52 Menampilkan Data Transaksi Berdasarkan Total Nilai Terbesar

3.5 *Transaction Control Language* (TCL)

Transaction Control Language (TCL) adalah sekumpulan perintah dalam SQL yang berfungsi sebagai manajer keamanan untuk operasi perubahan data (*INSERT*, *UPDATE*, *DELETE*) dalam basis data. Tujuan utama TCL adalah memastikan bahwa serangkaian modifikasi data (DML) harus selalu mengikuti prinsip ACID, di mana semua langkah perubahan (satu transaksi) harus sukses secara keseluruhan atau dibatalkan secara keseluruhan.

3.5.1 *Commit*

Perintah ini digunakan untuk menyimpan secara permanen semua perubahan yang telah dilakukan dalam transaksi saat ini ke dalam database. Setelah *COMMIT*, perubahan tidak dapat dibatalkan lagi

Contoh kode :

```
MariaDB [sate_rembige]> START TRANSACTION;

MariaDB [sate_rembige]> INSERT INTO item (id_item, item, harga)

   -> VALUES (10, 'Es Teh Manis', 800000.00);

MariaDB [sate_rembige]> UPDATE item SET harga = 2600000.00 WHERE id_item = 1;

Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [sate_rembige]> SELECT * FROM item WHERE id_item IN (1, 10);
```

Hasil :

```
MariaDB [sate_rembige]> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

MariaDB [sate_rembige]> INSERT INTO item (id_item, item, harga)
    -> VALUES (10, 'Es Teh Manis', 800000.00);
Query OK, 1 row affected (0.001 sec)

MariaDB [sate_rembige]> UPDATE item SET harga = 2600000.00 WHERE id_item = 1;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [sate_rembige]> SELECT * FROM item WHERE id_item IN (1, 10);
+---------+------------------+------------+
| id_item | item             | harga      |
+---------+------------------+------------+
|       1 | Sate Rembige Ayam | 2600000.00 |
|      10 | Es Teh Manis      |  800000.00 |
+---------+------------------+------------+
2 rows in set (0.000 sec)

MariaDB [sate_rembige]> COMMIT;
Query OK, 0 rows affected (0.005 sec)
```

Gambar 3. 53 Hasil Commit

3.5.2 *Rollback*

Perintah ini digunakan untuk membatalkan semua perubahan yang belum di-*commit* dan mengembalikan database ke kondisi terakhir saat *COMMIT* atau *START TRANSACTION* dilakukan.

Contoh kode :

```
MariaDB [sate_rembige]> START TRANSACTION;

MariaDB [sate_rembige]> INSERT INTO kasir (id_kasir, nama_kasir, TTD)

   -> VALUES (4, 'Dewi Sartika', 'ttd_dewi.png');

MariaDB [sate_rembige]> UPDATE kasir SET nama_kasir = 'Yuni Rahman' WHERE id_kasir = 3;

MariaDB [sate_rembige]> SELECT * FROM kasir WHERE id_kasir IN (3, 4);

MariaDB [sate_rembige]> ROLLBACK;

MariaDB [sate_rembige]> SELECT * FROM kasir WHERE id_kasir IN (3, 4);
```

Hasil :

```
MariaDB [sate_rembige]> START TRANSACTION;
Query OK, 0 rows affected (0.000 sec)

MariaDB [sate_rembige]> INSERT INTO kasir (id_kasir, nama_kasir, TTD)
    -> VALUES (4, 'Dewi Sartika', 'ttd_dewi.png');
Query OK, 1 row affected (0.001 sec)

MariaDB [sate_rembige]> UPDATE kasir SET nama_kasir = 'Yuni Rahman' WHERE id_kasir = 3;
Query OK, 1 row affected (0.001 sec)

MariaDB [sate_rembige]> SELECT * FROM kasir WHERE id_kasir IN (3, 4);
+----------+--------------+
| id_kasir | nama_kasir   |
+----------+--------------+
|        3 | Yuni Rahman  |
|        4 | Dewi Sartika |
+----------+--------------+
2 rows in set (0.000 sec)

MariaDB [sate_rembige]> ROLLBACK;
Query OK, 0 rows affected (0.004 sec)

MariaDB [sate_rembige]> SELECT * FROM kasir WHERE id_kasir IN (3, 4);
+----------+------------+
| id_kasir | nama_kasir |
+----------+------------+
|        3 | Yuni       |
+----------+------------+
1 row in set (0.000 sec)
```

Gambar 3. 54 Hasil Rollback

3.5.3 *Savepoint*

Perintah ini digunakan untuk menentukan titik simpan sementara dalam sebuah transaksi. Dengan adanya *savepoint*, Anda dapat melakukan *ROLLBACK* hanya pada sebagian transaksi, yaitu kembali ke titik simpan tertentu, tanpa membatalkan seluruh transaksi. Contoh kode :

MariaDB [sate_rembige]> START TRANSACTION;

MariaDB [sate_rembige]> SAVEPOINT savepoint_a;

MariaDB [sate_rembige]> INSERT INTO kasir (id_kasir, nama_kasir, TTD) VALUES (6, 'Rina Melati', 'ttd_rina.png');

MariaDB [sate_rembige]> SAVEPOINT savepoint_b;

MariaDB [sate_rembige]> UPDATE item SET harga = harga * 1.10 WHERE id_item = 1;

MariaDB [sate_rembige]> ROLLBACK TO savepoint_a;

MariaDB [sate_rembige]> COMMIT;

Hasil :

```
MariaDB [sate_rembige]> INSERT INTO kasir (id_kasir, nama_kasir, TTD) VALUES
    -> (6, 'Rina Melati', 'ttd_rina.png');
Query OK, 1 row affected (0.002 sec)

MariaDB [sate_rembige]> SAVEPOINT savepoint_b;
Query OK, 0 rows affected (0.001 sec)

MariaDB [sate_rembige]> UPDATE item SET harga = harga * 1.10 WHERE id_item = 1;
Query OK, 1 row affected (0.001 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [sate_rembige]> ROLLBACK TO savepoint_a;
Query OK, 0 rows affected (0.000 sec)

MariaDB [sate_rembige]> COMMIT;
Query OK, 0 rows affected (0.002 sec)
```

Gambar 3. 55 Hasil Savepoint

BAB IV
IMPLEMENTASI

4.1 Struktur Basis Data

Pada tahap implementasi system, perancangan basis data terdiri atas table – table yang saling terhubung melalui *primary key dan foreign key*. struktur database ini digunakan untuk menyimpan informasi terkait data konsumen, data kasir, jenis item, data transaksi dan detail transaksi.

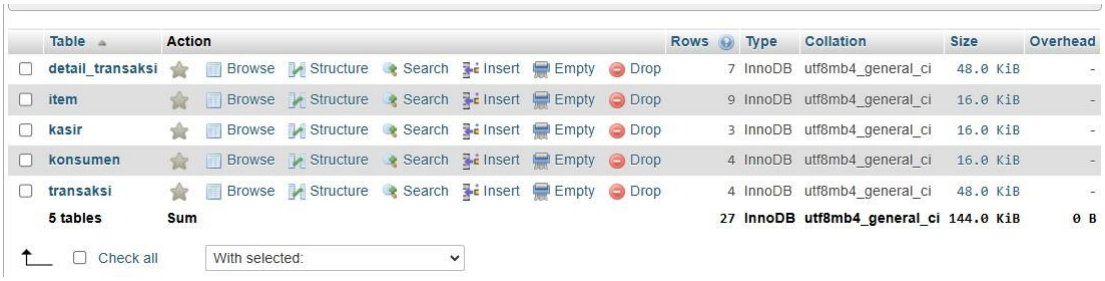


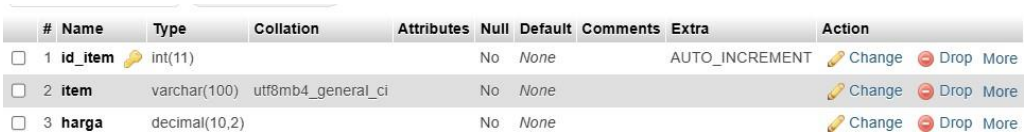
Table	Action	Rows	Type	Collation	Size	Overhead
detail_transaksi	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	48.0 KiB	-
item	Browse Structure Search Insert Empty Drop	9	InnoDB	utf8mb4_general_ci	16.0 KiB	-
kasir	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
konsumen	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	16.0 KiB	-
transaksi	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	48.0 KiB	-
5 tables	Sum	27	InnoDB	utf8mb4_general_ci	144.0 KiB	0 B

Check all With selected:

Gambar 4. 1 Daftar Tabel dalam Basis Data

4.1.1 Struktur Tabel Item

Tabel item digunakan untuk menyimpan data jenis-jenis item yang akan dijual di Sate Rembige. Tabel ini memiliki tiga kolom utama: id_item, yang berfungsi sebagai *Primary Key* untuk identifikasi unik setiap item; item, yang menyimpan nama jenis item dan harga, yang menyimpan harga jual dari item tersebut.



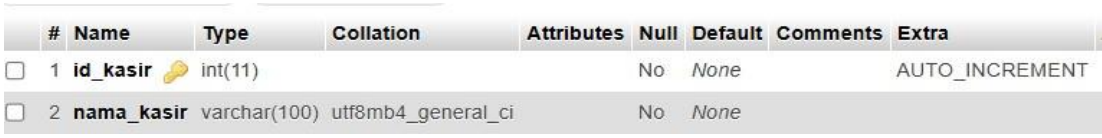
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id_item	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	item	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
3	harga	decimal(10,2)			No	None			Change Drop More

Gambar 4. 2 Struktur Tabel Item

4.1.2 Struktur Tabel Kasir

Tabel kasir berfungsi untuk menyimpan data identitas karyawan yang bertugas sebagai kasir. Kolom utamanya adalah id_kasir, yang merupakan *Primary Key* dan otomatis bertambah,

kolom nama_kasir untuk menyimpan nama lengkap kasir, serta kolom TTD (Tanda Tangan) yang opsional dan dapat dikosongkan.



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id_kasir	int(11)			No	None		AUTO_INCREMENT
2	nama_kasir	varchar(100)	utf8mb4_general_ci		No	None		

Gambar 4. 3 Struktur Tabel Kasir

4.1.3 Struktur Tabel Konsumen

Tabel konsumen berfungsi untuk menyimpan data identitas pelanggan atau konsumen yang melakukan pembelian. Tabel ini memiliki dua kolom, yaitu id_konsumen sebagai *Primary Key* yang otomatis bertambah, dan kolom nama_konsumen untuk mencatat nama lengkap konsumen.



Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id_konsumen	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	nama_konsumen	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More

Gambar 4. 4 Struktur Tabel Konsumen

4.1.4 Struktur Tabel Transaksi

Tabel transaksi menyimpan data ringkasan setiap penjualan yang terjadi. Tabel ini menggunakan id_transaksi sebagai *Primary Key* dan memiliki kolom kunci asing untuk mengidentifikasi konsumen (id_konsumen) dan kasir (id_kasir), yang keduanya opsional. Data transaksi mencakup tanggal transaksi, total harga (total), jumlah uang yang dibayar (dibayar), jumlah kembalian (kembalian), data pembayaran tunai, besaran pajak.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	id_transaksi	int(11)			No	None		AUTO_INCREMENT
2	id_konsumen	int(11)			Yes	NULL		
3	id_kasir	int(11)			Yes	NULL		
4	tanggal	date			No	None		
5	total	int(11)			No	None		
6	dibayar	int(11)			No	None		
7	kembalian	int(11)			No	None		
8	tunai	int(11)			Yes	NULL		
9	pajak	decimal(10,2)			Yes	NULL		

Gambar 4. 5 Struktur Tabel Transaksi

4.1.5 Struktur Tabel Detail_Transaksi

Tabel ini menyimpan rincian item yang termasuk dalam sebuah transaksi, yang menghubungkan id_transaksi dan id_item (keduanya adalah kunci asing) yang opsional. Rincian tersebut mencakup jumlah kuantitas item yang dibeli (qty) dan subtotal harga untuk item tersebut (subtotal), di mana kedua kolom ini wajib diisi dan tidak boleh kosong.

Table structure | Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id_transaksi	int(11)			Yes	NULL			Change Drop More
2	id_item	int(11)			Yes	NULL			Change Drop More
3	qty	int(11)			No	None			Change Drop More
4	subtotal	int(11)			No	None			Change Drop More

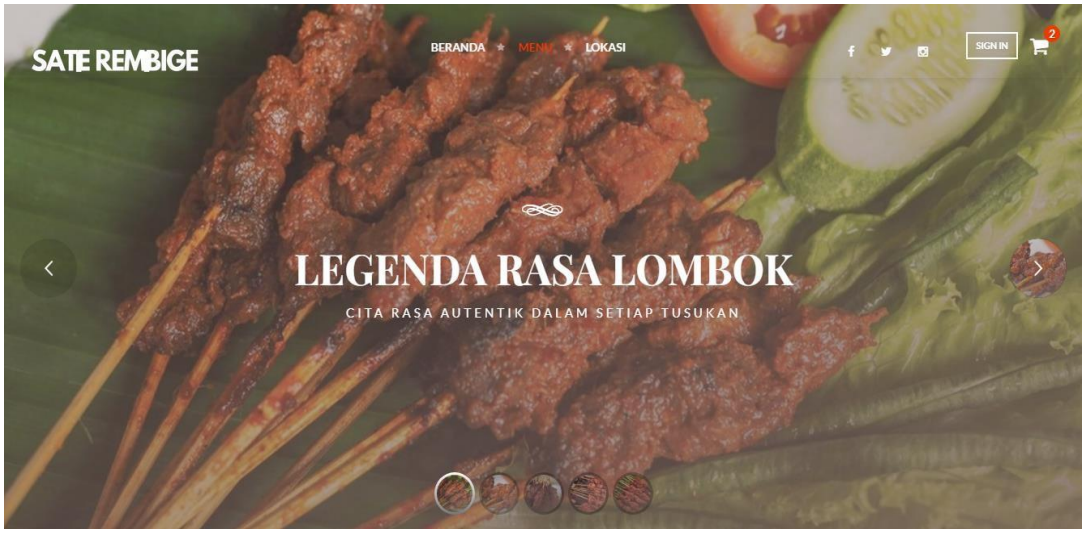
Gambar 4. 6 struktur detail_transaksi

4.2 Antarmuka Sistem

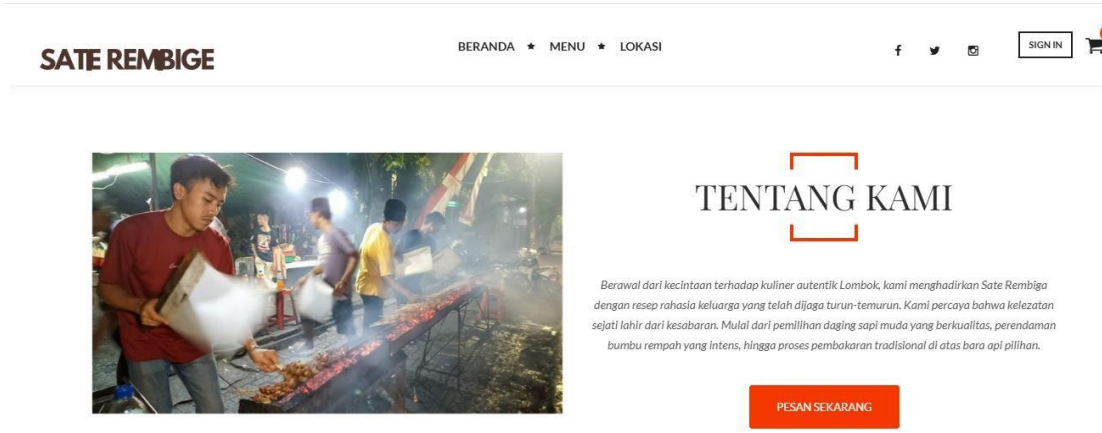
User Interface (UI) adalah tampilan visual dari sebuah sistem atau aplikasi yang digunakan untuk berinteraksi dengan pengguna. UI mencakup elemen seperti tombol, menu, ikon, warna, tipografi, dan tata letak yang dirancang agar pengguna dapat memahami dan menggunakan sistem dengan mudah. Tujuan utama UI adalah menciptakan tampilan yang menarik, jelas, dan intuitif sehingga pengguna tidak kesulitan saat menjalankan aplikasi.

4.2.1 Halaman Index

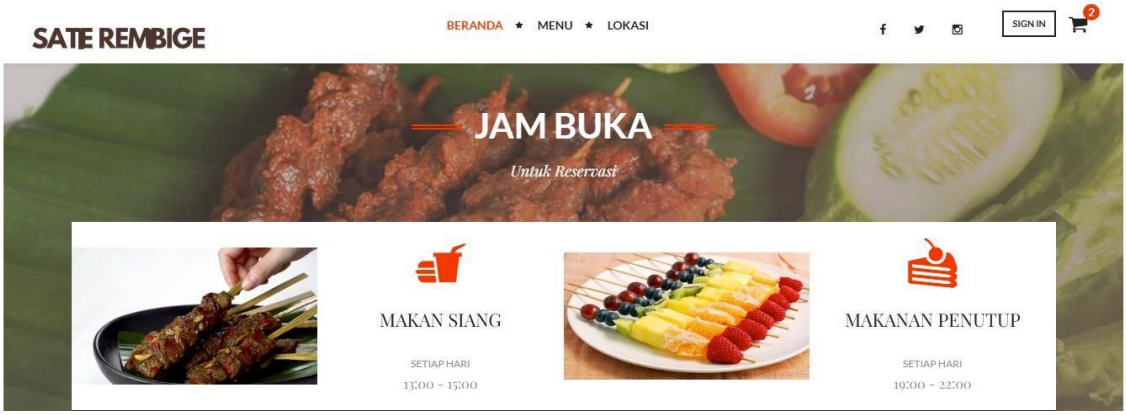
Halaman index merupakan antarmuka utama yang pertama kali dimuat oleh peramban saat pengguna mengakses alamat domain website.



Gambar 4. 7 index 1



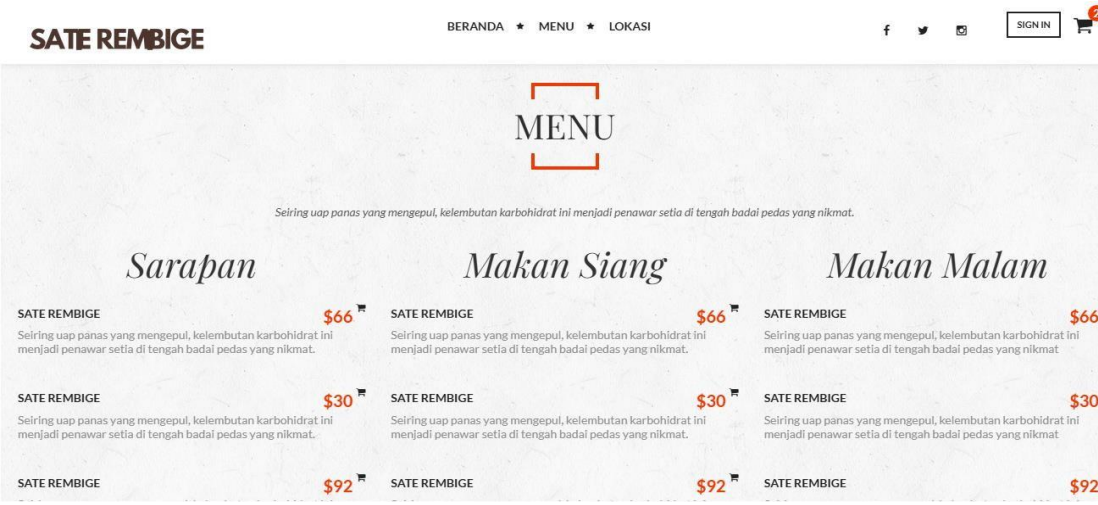
Gambar 4. 8 index 2



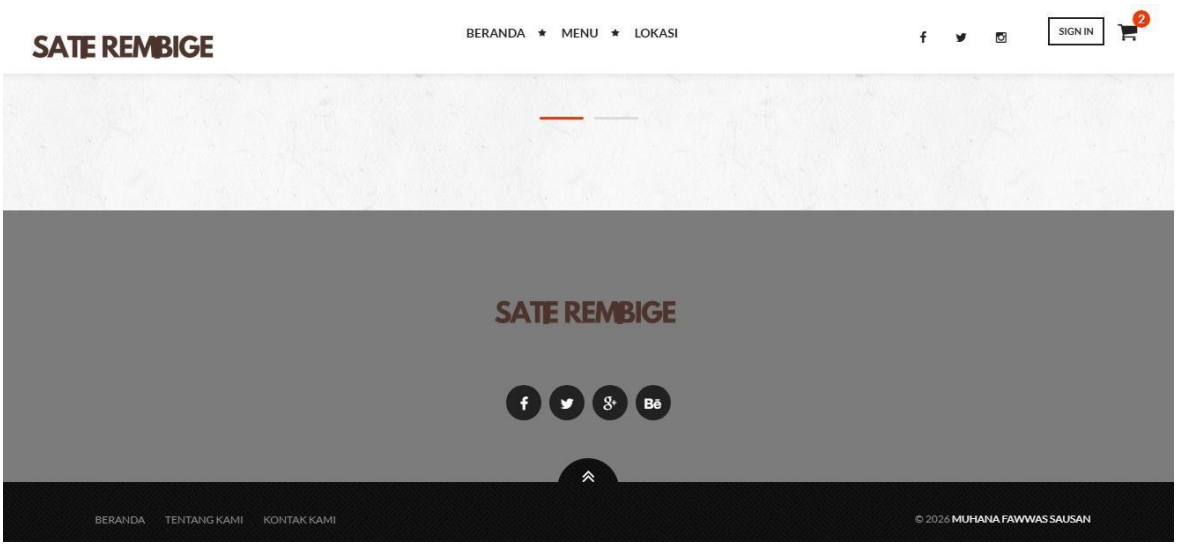
Gambar 4. 9 index 3



Gambar 4. 10 index 4



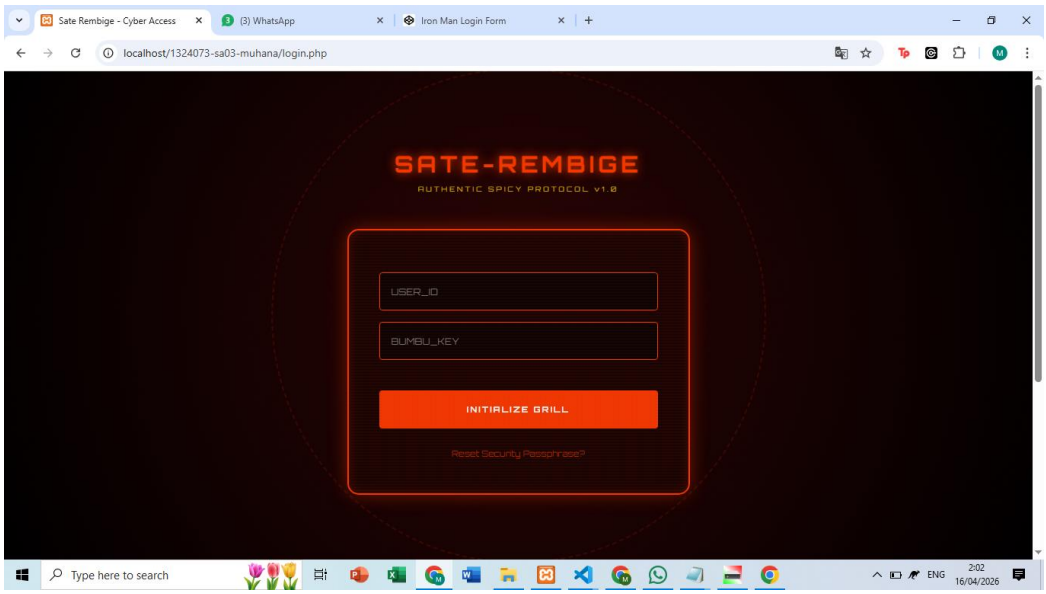
Gambar 4. 11 index 5



Gambar 4. 12 index 6

4.2.2 Halaman Login

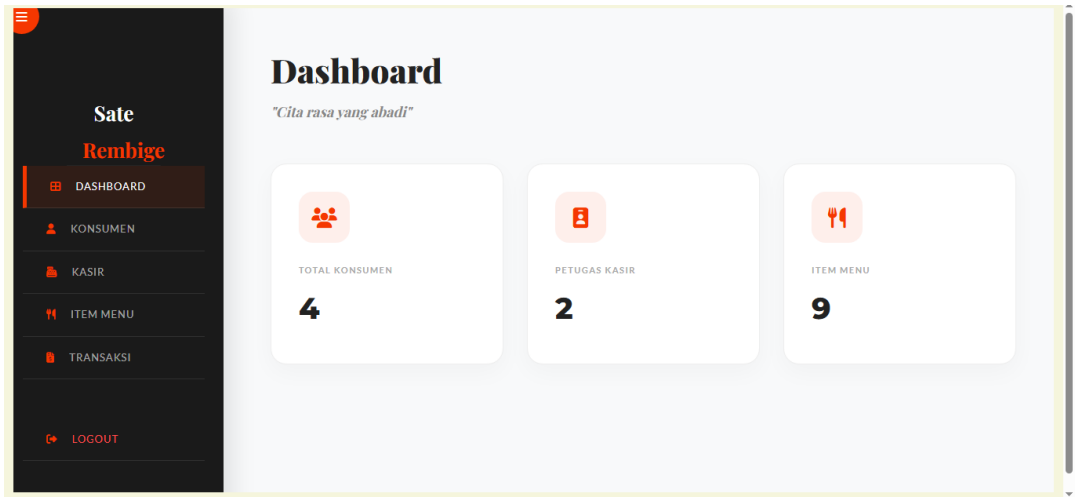
Halaman login merupakan halaman awal yang digunakan untuk mengakses sistem. Pada halaman ini, pengguna diminta untuk memasukkan username dan password yang telah terdaftar. Sistem akan melakukan validasi terhadap data yang dimasukkan. Apabila data sesuai, maka pengguna akan diarahkan ke halaman dashboard. Namun, jika terjadi kesalahan input, sistem akan menampilkan pesan error sebagai peringatan. Halaman ini berfungsi untuk menjaga keamanan sistem agar hanya pengguna tertentu yang dapat mengakses aplikasi.



Gambar 4. 13 login

4.2.3 Halaman Dashboard

Halaman dashboard merupakan halaman utama setelah pengguna berhasil melakukan login. Halaman ini berfungsi sebagai pusat navigasi yang menampilkan menu-menu utama seperti data kasir, konsumen, item, dan transaksi. Selain itu, dashboard juga memberikan gambaran umum terkait aktivitas sistem, sehingga memudahkan pengguna dalam mengakses fitur yang dibutuhkan secara cepat dan efisien.



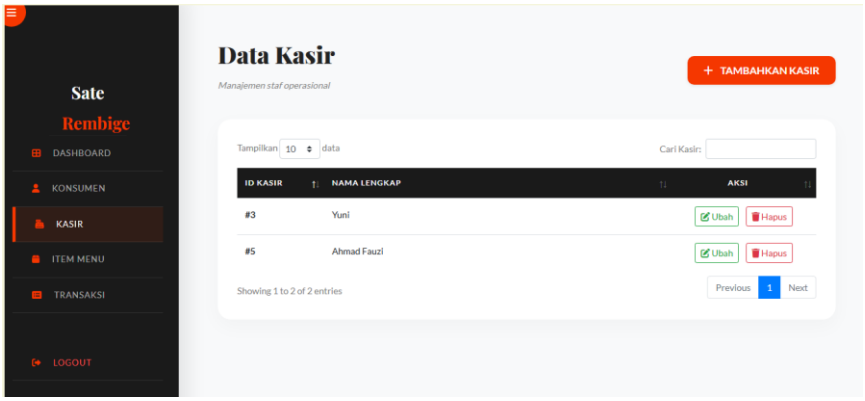
Gambar 4. 14 dashboard

4.2.4 Halaman Master Kasir

Halaman master kasir digunakan untuk mengelola data kasir yang terdaftar dalam sistem. Pada halaman ini, pengguna dapat melakukan proses tambah, ubah, dan hapus data kasir. Data yang dikelola meliputi identitas kasir seperti nama dan informasi lainnya. Dengan adanya fitur ini, sistem dapat mencatat siapa saja yang bertugas dalam melakukan transaksi.

4.2.4.1 Halaman Kasir Lihat

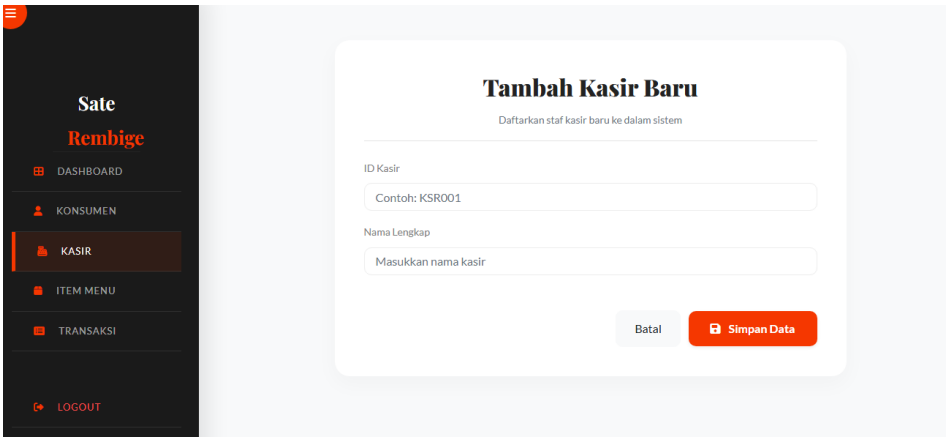
Halaman ini menampilkan daftar seluruh petugas kasir yang memiliki hak akses ke sistem Sate Rembige. Informasi yang disajikan mencakup nama petugas, *username*, dan identitas unik lainnya. Melalui halaman ini, admin dapat memantau siapa saja personel yang berwenang menjalankan operasional harian.



Gambar 4. 15 kasir lihat

4.2.4.2 Halaman Kasir Tambah

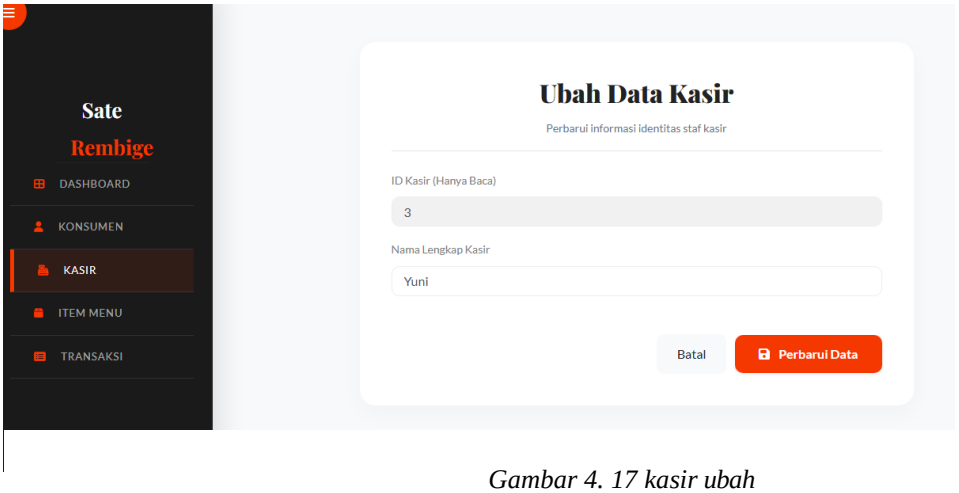
Halaman ini digunakan untuk mendaftarkan petugas kasir baru. Selain menginputkan nama lengkap, admin juga menentukan *username* dan *password* untuk petugas tersebut. Data ini akan disimpan ke dalam tabel kasir dan digunakan sebagai kredensial saat petugas melakukan proses login ke sistem.



Gambar 4. 16 kasir tambah

4.2.4.3 Halaman Kasir Ubah

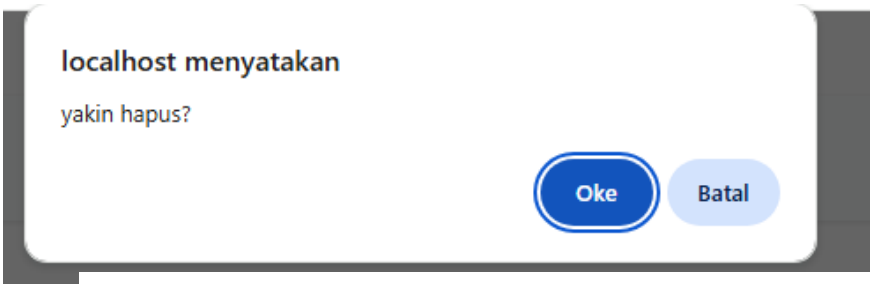
Halaman ini berfungsi untuk memperbarui profil petugas kasir, seperti perubahan nama atau pembaruan kata sandi (*password*) secara berkala. Fitur ubah ini sangat penting untuk menjaga aspek keamanan akun petugas guna menghindari penyalahgunaan hak akses oleh pihak yang tidak bertanggung jawab.



Gambar 4. 17 kasir ubah

4.2.4.4 Halaman Kasir Hapus

Halaman ini digunakan untuk mencabut hak akses petugas kasir dari sistem, misalnya apabila petugas tersebut sudah tidak bekerja lagi. Dengan menghapus data kasir, maka secara otomatis kredensial login petugas tersebut tidak akan bisa digunakan kembali, sehingga integritas sistem tetap terjaga.



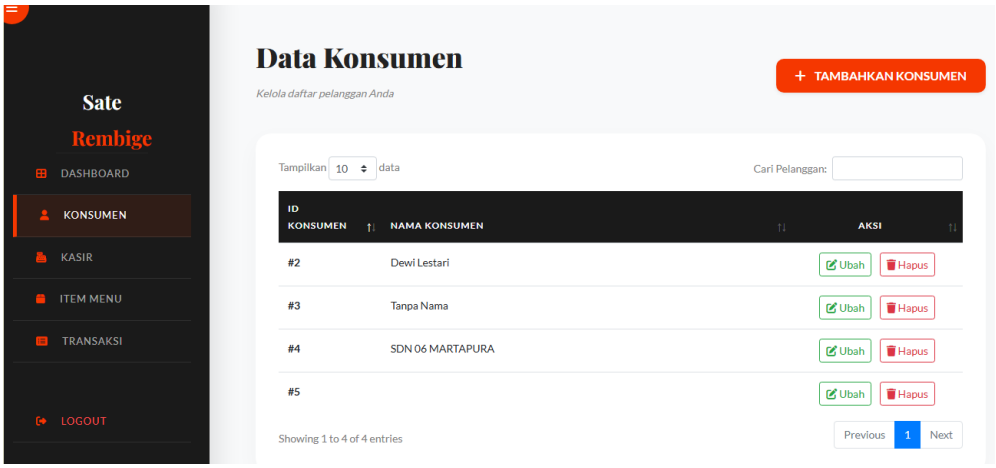
Gambar 4. 18 kasir hapus

4.2.5 Halaman Master Konsumen

Halaman master konsumen berfungsi untuk mengelola data konsumen. Pengguna dapat menambahkan data konsumen baru, mengubah data yang sudah ada, serta menghapus data yang tidak diperlukan. Data konsumen yang tersimpan akan digunakan dalam proses transaksi sehingga memudahkan pencatatan dan pelacakan riwayat transaksi berdasarkan konsumen.

.4.2.5.1Halaman Konsumen Lihat

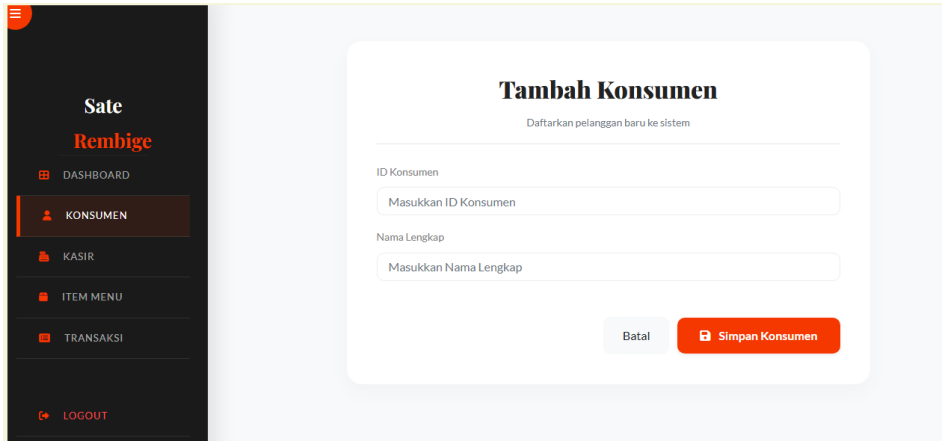
Halaman ini menampilkan tabel data pelanggan yang berisi informasi seperti ID Konsumen, nama lengkap. Dengan tampilan yang bersih, admin dapat dengan mudah mencari dan melihat daftar pelanggan yang sudah terdaftar di dalam sistem.



Gambar 4. 19 konsumen lihat

4.2.5.2 Halaman Konsumen Tambah

Halaman ini menyediakan formulir untuk mendaftarkan pelanggan baru. Informasi yang diinputkan akan disimpan ke dalam tabel konsumen. Fitur ini biasanya digunakan oleh kasir sebelum memulai transaksi jika pelanggan tersebut belum pernah terdaftar sebelumnya di sistem.

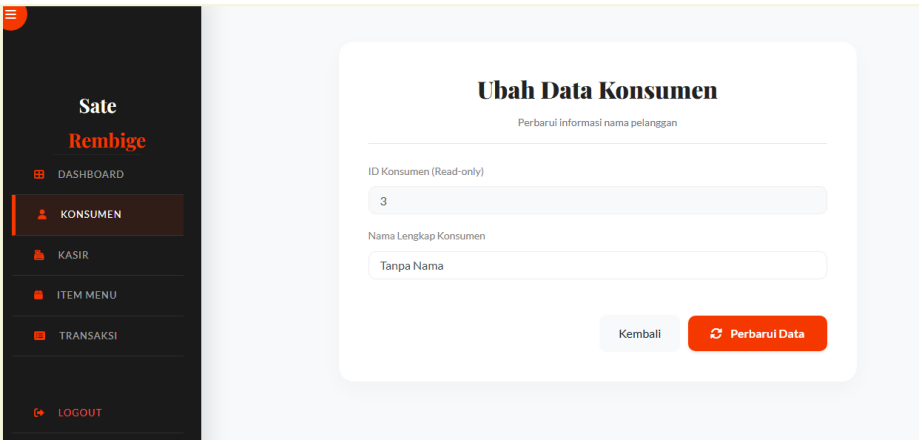


Gambar 4. 20 konsumen tambah

4.2.5.3 Halaman Konsumen Ubah

Halaman ini berfungsi untuk memperbarui informasi detail pelanggan, seperti perubahan nama yang masih aktif. Hal ini memastikan data yang tersimpan di

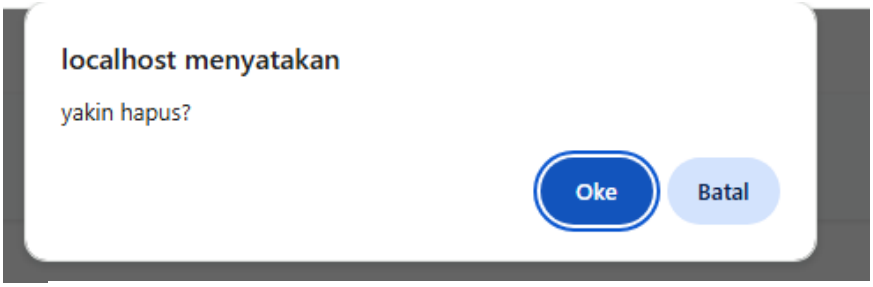
dalam basis data tetap aktual dan valid untuk kebutuhan operasional maupun komunikasi di masa mendatang.



Gambar 4. 21 konsumen ubah

4.2.5.4 Halaman Konsumen Hapus

Halaman ini digunakan untuk membersihkan data pelanggan yang sudah tidak relevan atau terjadi kesalahan input yang tidak bisa diperbaiki. Proses penghapusan data ini dilengkapi dengan mekanisme konfirmasi guna menjaga keamanan data agar tidak terhapus secara permanen secara tidak sengaja.



Gambar 4. 22 konsumen hapus

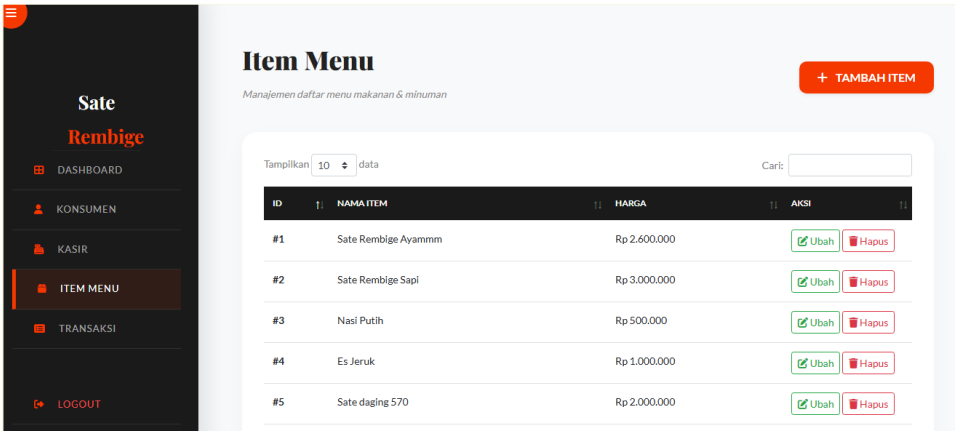
4.2.6 Halaman Master Item

Halaman master item digunakan untuk mengelola data barang atau jasa yang tersedia. Pada halaman ini, pengguna dapat menambahkan item baru, mengubah informasi item, serta menghapus item yang tidak digunakan. Data yang disimpan meliputi

nama item dan harga. Data item ini akan digunakan dalam proses transaksi untuk menentukan jumlah pembayaran.

4.2.6.1 Halaman Item Lihat

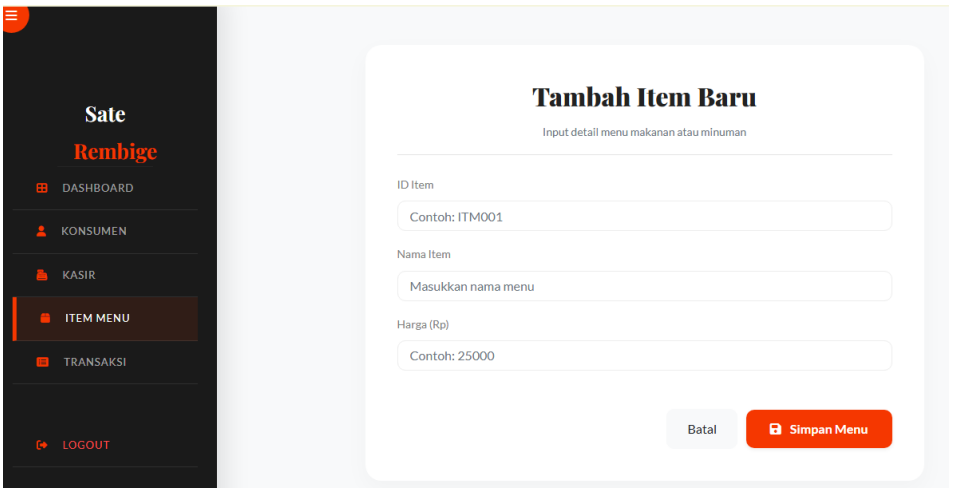
Halaman ini menampilkan daftar seluruh katalog menu dalam bentuk tabel yang informatif. Data yang ditampilkan meliputi kode item, nama menu (seperti Sate Rembige, Lontong, dll.), serta harga satuan. Fitur ini memudahkan admin untuk mengecek ketersediaan menu dan memantau harga jual yang sedang berlaku.



Gambar 4. 23 item lihat

4.2.6.2 Halaman Item Tambah

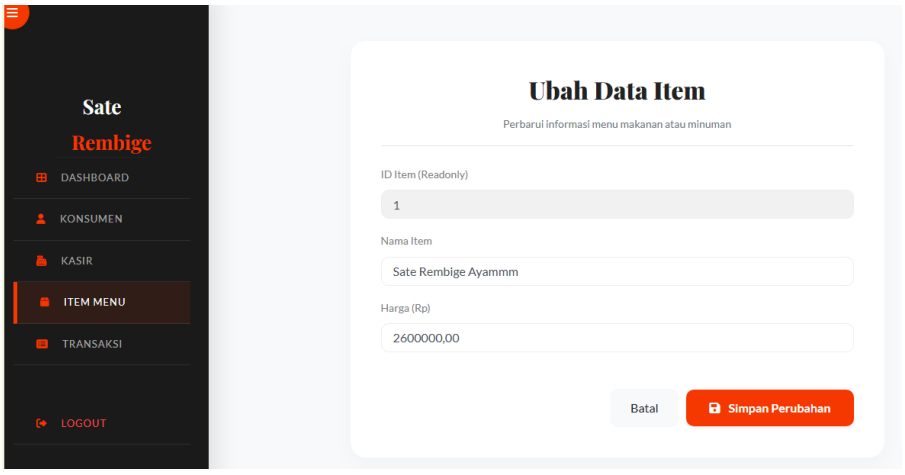
Halaman ini digunakan untuk menambahkan menu baru ke dalam sistem. Pengguna dapat menginputkan nama item dan menentukan harga jualnya. Data yang diinputkan melalui formulir ini akan langsung masuk ke tabel item di basis data dan secara otomatis muncul sebagai pilihan saat kasir melakukan transaksi.



Gambar 4. 24 item tambah

4.2.6.3 Halaman Item Ubah

Halaman ini berfungsi untuk melakukan pembaruan informasi pada menu yang sudah ada. Fitur ini sangat penting jika terjadi perubahan harga pasar atau revisi nama menu. Perubahan harga pada halaman ini tidak akan mengubah riwayat transaksi lama, namun akan berlaku untuk transaksi-transaksi yang dilakukan setelah perubahan disimpan.

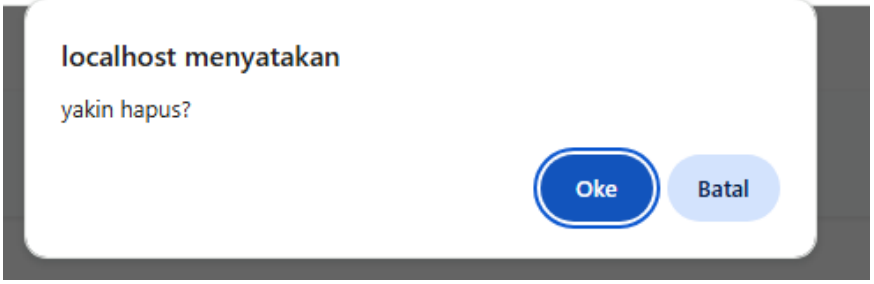


Gambar 4. 25 item ubah

4.2.6.4 Halaman Item Hapus

Halaman ini disediakan untuk menghapus menu yang sudah tidak lagi dijual atau menu yang salah input. Sistem biasanya akan memberikan peringatan konfirmasi sebelum

data benar-benar dihapus untuk mencegah hilangnya data menu secara tidak sengaja yang dapat memengaruhi integritas relasi data.



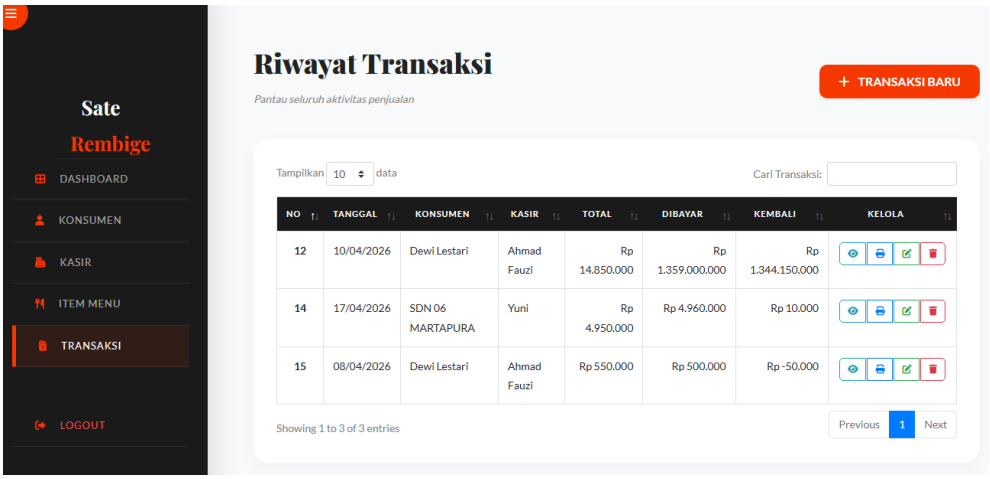
Gambar 4. 26 item hapus

4.2.7 Halaman Transaksi

Halaman transaksi merupakan bagian utama dalam sistem yang digunakan untuk melakukan pencatatan transaksi. Pada halaman ini, pengguna dapat memilih konsumen dan kasir, serta memasukkan detail transaksi seperti item yang dibeli, jumlah, dan total harga. Sistem secara otomatis akan menghitung total pembayaran, pajak, serta kembalian berdasarkan jumlah uang yang dibayarkan. Selain itu, pengguna juga dapat melihat detail transaksi, mengubah, menghapus, serta mencetak bukti transaksi. Halaman ini berfungsi untuk memastikan seluruh proses transaksi tercatat dengan baik dan terorganisir.

4.2.7.1 Halaman Transaksi Lihat

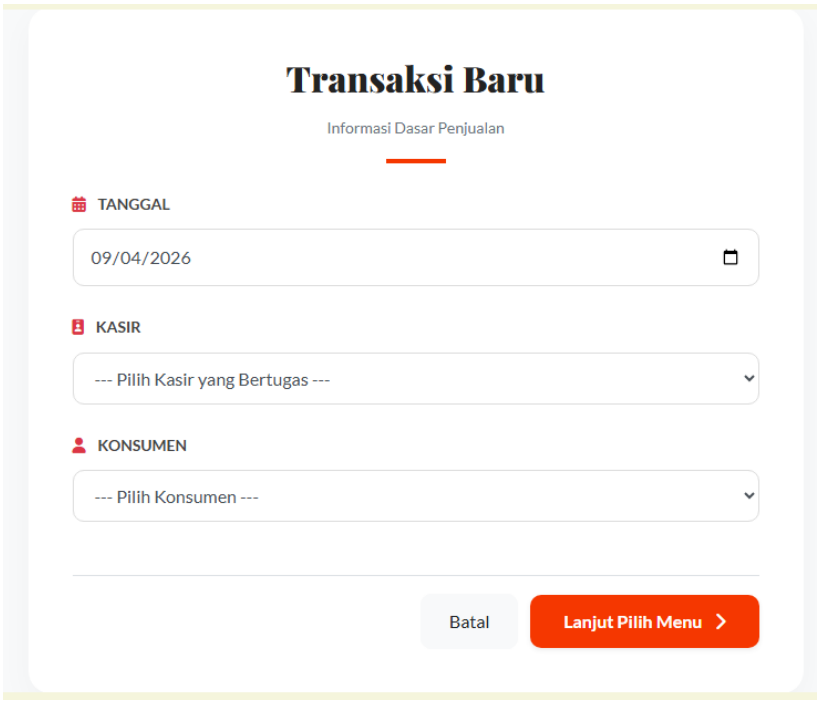
Halaman ini menampilkan tabel ringkasan seluruh riwayat transaksi yang telah tersimpan dalam basis data. Informasi yang disajikan mencakup ID Transaksi, nama konsumen, nama kasir yang bertugas, tanggal transaksi, total bayar, hingga status pembayaran. Halaman ini memudahkan admin untuk memantau arus penjualan secara keseluruhan.



Gambar 4. 27 transaksi lihat

4.2.7.2 Halaman Transaksi Tambah

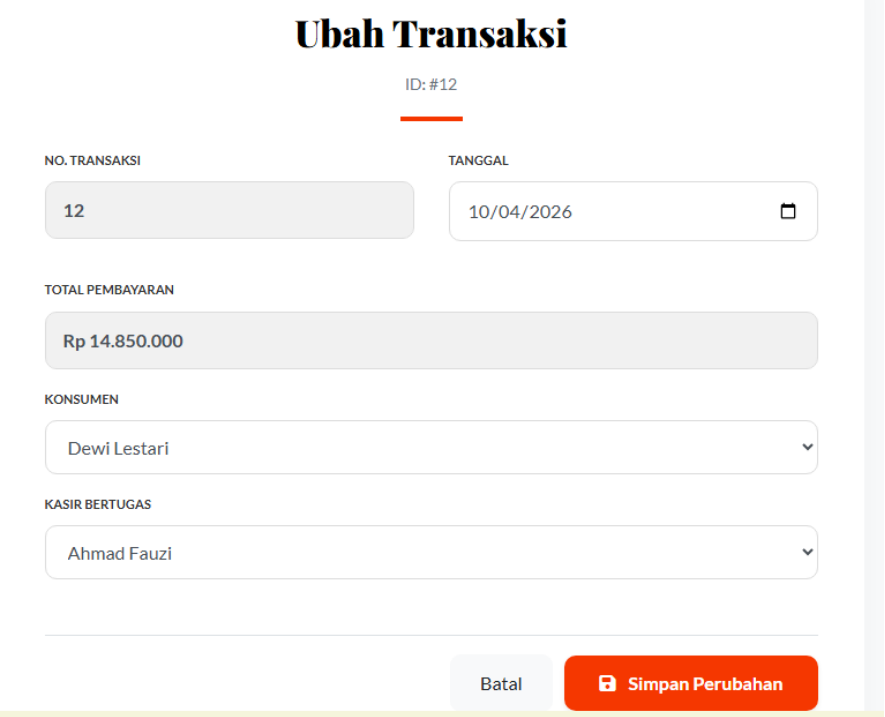
Halaman ini berfungsi untuk membuka sesi transaksi baru. Pada antarmuka ini, kasir melakukan input data awal seperti pemilihan nama konsumen dan penentuan tanggal. Setelah data utama ini disimpan, sistem akan menghasilkan ID Transaksi unik yang nantinya akan digunakan untuk menghubungkan data induk dengan detail item yang dibeli.



Gambar 4. 28 transaksi tambah

4.2.7.3 Halaman Transaksi Ubah

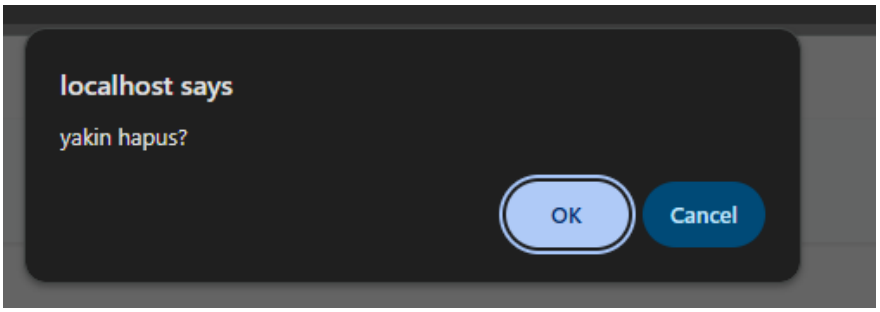
Halaman ini digunakan untuk memperbarui informasi pada header transaksi jika terdapat kekeliruan data non-item, seperti salah memilih nama konsumen atau kesalahan input jumlah uang tunai yang diterima. Fitur ini memastikan bahwa informasi administratif pada nota tetap akurat sesuai dengan kondisi lapangan.



Gambar 4. 29 transaksi ubah

4.2.7.4 Halaman Transaksi Hapus

Halaman ini berfungsi untuk membatalkan seluruh rangkaian transaksi. Apabila sebuah data transaksi dihapus melalui halaman ini, maka sistem secara otomatis akan menghapus data terkait yang ada pada tabel detail transaksi (cascade delete/manual clean-up). Hal ini bertujuan untuk menjaga integritas basis data agar tidak ada data detail yang "tertinggal" tanpa memiliki induk transaksi.



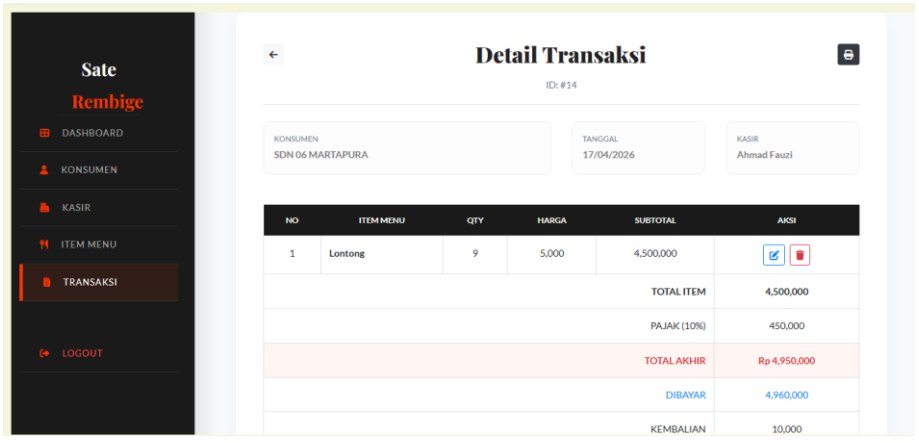
Gambar 4. 30 transaksi hapus

4.2.8 Halaman Detail Transaksi

Halaman ini adalah inti dari proses bisnis "Sate Rembige". Pada bagian ini, sistem mencatat pesanan, menghitung subtotal secara otomatis berdasarkan kuantitas yang diinput, serta menghitung pajak dan total bayar. Halaman ini menghubungkan entitas kasir, konsumen, dan item dalam satu rekaman transaksi yang sah.

4.2.8.1 Halaman Detail Transaksi Lihat

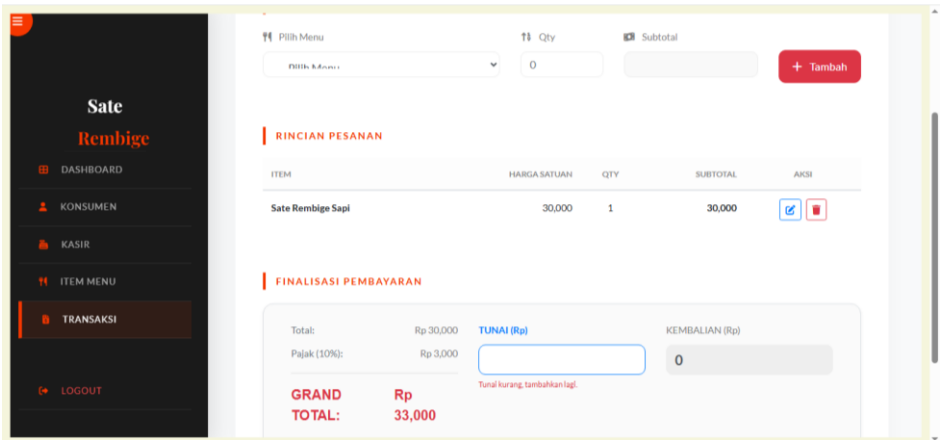
Halaman ini menyajikan daftar seluruh item yang telah dipilih dalam satu transaksi tertentu. Informasi yang ditampilkan meliputi nama item, harga satuan, kuantitas yang dibeli, dan subtotal per item. Halaman ini juga merangkum total belanja, perhitungan pajak (10%), serta status pembayaran (tunai dan kembalian).



Gambar 4. 31 transaksi detail lihat

4.2.8.2 Halaman Detail Transaksi Tambah

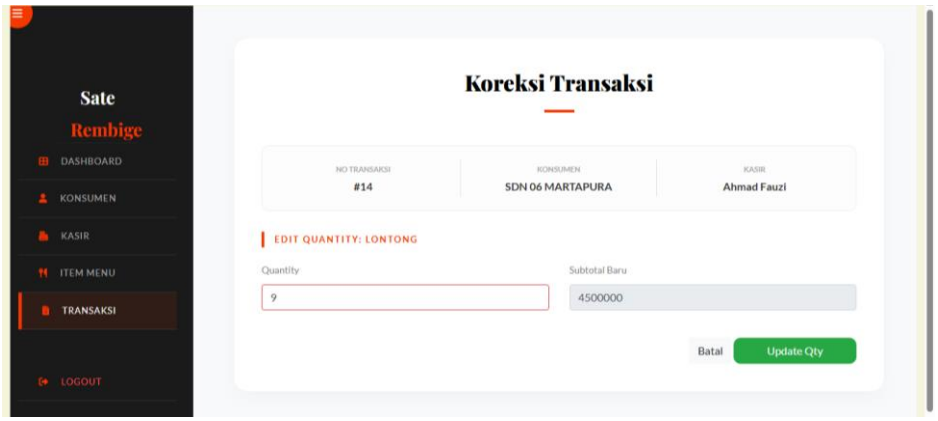
Halaman ini digunakan untuk memasukkan item baru ke dalam transaksi yang sedang berjalan. Kasir dapat memilih item dari daftar menu yang tersedia dan menentukan jumlah unit (*quantity*) yang dipesan. Sistem secara otomatis akan melakukan perhitungan subtotal berdasarkan harga master item sebelum disimpan ke dalam tabel detail transaksi.



Gambar 4. 32 transaksi detail tambah

4.2.8.3 Halaman Detail Transaksi Ubah

Halaman ini berfungsi sebagai fitur koreksi apabila terjadi kesalahan input jumlah barang setelah transaksi tercatat. Melalui antarmuka ini, pengguna dapat memperbarui kuantitas item. Setelah data diperbarui, sistem secara otomatis akan melakukan kalkulasi ulang terhadap total harga, pajak, dan nilai kembalian pada nota tersebut untuk menjaga akurasi data keuangan.



Gambar 4. 33 *transaksi detail ubah*

4.2.8.4 Halaman Detail Transaksi Hapus

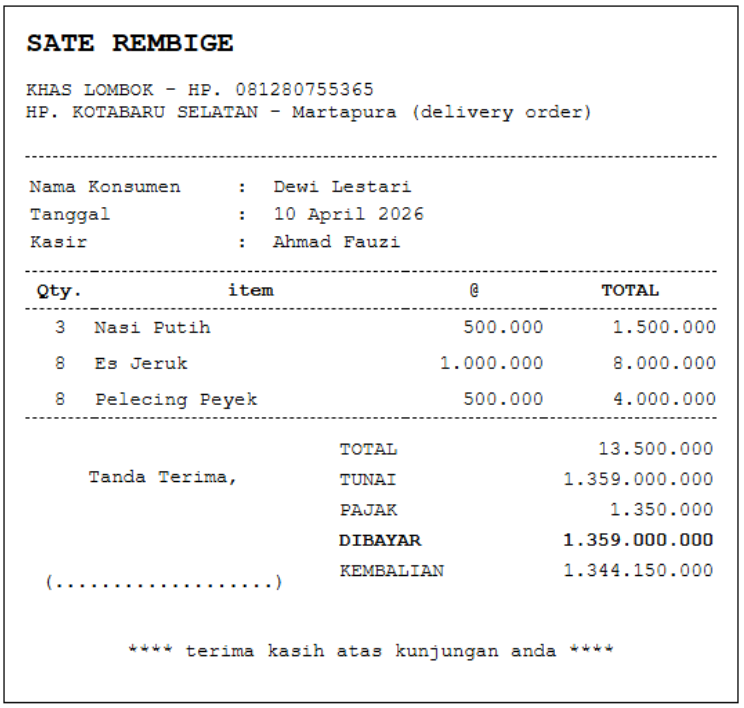
Halaman ini digunakan untuk membatalkan atau menghapus salah satu item dari daftar belanja jika konsumen membatalkan pesanan untuk item tertentu. Proses penghapusan ini diikuti dengan pembersihan data di tabel detail transaksi dan pembaruan nilai total pada tabel transaksi utama guna memastikan sinkronisasi data tetap terjaga.



Gambar 4. 34 transaksi detail hapus

4.2.8 Halaman Cetak

Halaman ini merupakan *output* akhir dari sistem informasi Sate Rembige. Fungsi utamanya adalah menghasilkan dokumen fisik atau digital (PDF) sebagai bukti transaksi yang sah untuk diserahkan kepada konsumen.



SATE REMBIGE

KHAS LOMBOK - HP. 081280755365
HP. KOTABARU SELATAN - Martapura (delivery order)

Nama Konsumen : Dewi Lestari
Tanggal : 10 April 2026
Kasir : Ahmad Fauzi

Qty.	item	@	TOTAL
3	Nasi Putih	500.000	1.500.000
8	Es Jeruk	1.000.000	8.000.000
8	Pelecing Peyek	500.000	4.000.000

	TOTAL	13.500.000
Tanda Terima,	TUNAI	1.359.000.000
	PAJAK	1.350.000
	DIBAYAR	**1.359.000.000**
(.....)	KEMBALIAN	1.344.150.000

**** terima kasih atas kunjungan anda ****

Gambar 4. 35 cetak

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa pembangunan sistem informasi penjualan "Sate Rembige" telah berhasil mentransformasi proses pencatatan transaksi dari metode konvensional ke dalam bentuk digital yang lebih terstruktur. Sistem ini mampu menjamin integritas data melalui relasi antar tabel yang solid, mulai dari data master kasir, konsumen, hingga item menu. Efisiensi operasional meningkat secara signifikan berkat fitur kalkulasi otomatis pada perhitungan subtotal, pajak, dan kembalian, yang secara langsung meminimalisir risiko kesalahan manusia (*human error*). Selain itu, fleksibilitas dalam pengelolaan transaksi yang ditawarkan oleh fitur koreksi pada detail transaksi memberikan kemudahan bagi pengguna dalam menjaga akurasi laporan keuangan tanpa harus mengulang proses input dari awal.

5.2 Saran

Meskipun sistem telah berfungsi dengan baik sesuai kebutuhan utama, pengembangan lebih lanjut sangat disarankan untuk meningkatkan nilai guna aplikasi di masa depan. Penguatan aspek keamanan dapat dilakukan dengan mengimplementasikan enkripsi data yang lebih kompleks serta pembagian level hak akses pengguna yang lebih spesifik. Dari sisi fungsionalitas, sistem ini dapat dikembangkan lebih lanjut dengan menambahkan modul manajemen stok barang secara *real-time* dan fitur laporan penjualan periodik yang dapat diekspor ke berbagai format dokumen. Terakhir, pengembangan antarmuka yang lebih responsif juga diperlukan agar aplikasi dapat diakses secara optimal melalui berbagai perangkat, baik *desktop* maupun *mobile*, guna mendukung mobilitas kasir di lapangan.

DAFTAR PUSTAKA

Connolly, T. M., & Begg, C. E. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Essex: Pearson Education Limited.

Hariyanto, B. (2004). *Sistem Manajemen Basis Data*. Bandung: Informatika.

Kadir, A. (2021). *Belajar Sendiri Database MySQL*. Yogyakarta: Andi Offset.

MariaDB Foundation. (n.d.). *MariaDB Knowledge Base: SQL Statements (DDL, DML, DCL, TCL)*. Diakses dari https://mariadb.com/kb/en/sql-statements/.

Sate Rembige. (2015). *Struk Transaksi Penjualan: SDN 06 MARTAPURA (05 Desember 2015)*.